MASTER IN ASTROPHYSICS
AND SPACE SCIENCE

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Optimisation & Monte Carlo Sampling

Introduction to Numerical Methods for Astrophysics

Fritz Ali Agildere

November 27, 2025

Overview

Overview

Overview

Frequentist View

Frequentist View

- *With Prior Knowledge:* out of all n different but equally likely possible outcomes of an event, there are k that have a property A which is therefore measured with probability

$$P(A) = \frac{k}{n}$$

Frequentist View

- *With Prior Knowledge:* out of all n different but equally likely possible outcomes of an event, there are k that have a property A which is therefore measured with probability

$$P(A) = \frac{k}{n}$$

- *Without Prior Knowledge:* out of n independent measurements the outcome A occurs k times, defining

$$P(A) = \lim_{n \to \infty} \frac{k}{n}$$

Bayesian View

Bayesian View

- $P(A|B)$ quantifies the plausibility of an assumption A under known information given by B

Bayesian View

- $P(A|B)$ quantifies the plausibility of an assumption A under known information given by B
- *Kolmogorov Axioms* underpin the rules for calculating with probabilities

Bayesian View

- $P(A|B)$ quantifies the plausibility of an assumption A under known information given by B
- *Kolmogorov Axioms* underpin the rules for calculating with probabilities
- $P(A, B) = P(A)\,P(B|A)$ quantifies the probability of measuring A and B simultaneously, and from assuming $P(A, B) = P(B, A)$ we find

Bayesian View

- $P(A|B)$ quantifies the plausibility of an assumption A under known information given by B
- *Kolmogorov Axioms* underpin the rules for calculating with probabilities
- $P(A,B) = P(A)\,P(B|A)$ quantifies the probability of measuring A and B simultaneously, and from assuming $P(A,B) = P(B,A)$ we find

Bayes' Theorem

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)}$$

Continuous Distributions

Continuous Distributions

- *Cumulative Distribution Function:* $F(x) = P(X \leq x)$

Continuous Distributions

- *Cumulative Distribution Function:* $F(x) = P(X \leq x)$
- *Probability Density Function:* $f(x) = \frac{d}{dx}F(x)$

Continuous Distributions

- *Cumulative Distribution Function:* $F(x) = P(X \leq x)$
- *Probability Density Function:* $f(x) = \frac{d}{dx}F(x)$

$$P(a \leq X \leq b) = \int_a^b f(x)\,dx = F(b) - F(a)$$

Continuous Distributions

- *Cumulative Distribution Function:* $F(x) = P(X \leq x)$
- *Probability Density Function:* $f(x) = \dfrac{d}{dx}F(x)$

$$P(a \leq X \leq b) = \int_a^b f(x)\,dx = F(b) - F(a)$$

- *Normalization:*

Continuous Distributions

- *Cumulative Distribution Function:* $F(x) = P(X \le x)$
- *Probability Density Function:* $f(x) = \dfrac{d}{dx} F(x)$

$$P(a \le X \le b) = \int_a^b f(x)\,dx = F(b) - F(a)$$

- *Normalization:* $1 = \int_{-\infty}^{+\infty} f(x)\,dx$

Continuous Distributions

- *Cumulative Distribution Function:* $F(x) = P(X \leq x)$
- *Probability Density Function:* $f(x) = \frac{d}{dx}F(x)$

$$P(a \leq X \leq b) = \int_a^b f(x)\,dx = F(b) - F(a)$$

- *Normalization:* $1 = \int_{-\infty}^{+\infty} f(x)\,dx \qquad \lim_{x \to -\infty} F(x) = 0 \qquad \lim_{x \to +\infty} F(x) = 1$

Expectation Values

Expectation Values

$$E[h(x)] = \int_{-\infty}^{+\infty} h(x)\, f(x)\, dx$$

Expectation Values

$$E[h(x)] = \int_{-\infty}^{+\infty} h(x)\, f(x)\, dx$$

- *Mean* or first raw moment: $\mu = E[X] = \int_{-\infty}^{+\infty} x f(x)\, dx$

Expectation Values

$$E[h(x)] = \int_{-\infty}^{+\infty} h(x)\, f(x)\, dx$$

- *Mean* or first raw moment: $\mu = E[X] = \int_{-\infty}^{+\infty} x f(x)\, dx$
- *Variance* or second raw moment: $\sigma^2 = E\left[(X - E[X])^2\right] = \int_{-\infty}^{+\infty} (x - E[x])^2 f(x)\, dx$

Overview

Drawing Numbers

Drawing Numbers

$$x \sim U(a, b) \qquad\qquad y \sim N(\mu, \sigma)$$

Drawing Numbers

$$x \sim U(a, b) \qquad\qquad y \sim N(\mu, \sigma)$$

- *Uniform Distribution:* $U(a, b) = f(x|a, b) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$

Drawing Numbers

$$x \sim U(a, b) \qquad\qquad y \sim N(\mu, \sigma)$$

- *Uniform Distribution:* $U(a, b) = f(x|a, b) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$
- *Gaussian Distribution:* $N(\mu, \sigma) = f(y|\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\left(\frac{y-\mu}{\sigma}\right)^2\right)$

Number Generation

Number Generation

- *Pseudo Random Numbers:* $x_{j+1} = g(x_1, \dots, x_j)$

Number Generation

- *Pseudo Random Numbers:* $x_{j+1} = g(x_1, \ldots, x_j)$
  - deterministic & reproducible

Number Generation

- *Pseudo Random Numbers:* $x_{j+1} = g(x_1, \dots, x_j)$
  - deterministic & reproducible
  - efficient & transparent

Number Generation

- *Pseudo Random Numbers:* $x_{j+1} = g(x_1, \ldots, x_j)$
  - deterministic & reproducible
  - efficient & transparent
  - almost indistinguishable with long periods of recurrence

Number Generation

- *Pseudo Random Numbers:* $x_{j+1} = g(x_1, \dots, x_j)$
  - deterministic & reproducible
  - efficient & transparent
  - almost indistinguishable with long periods of recurrence
- *Equal Distribution Generators:*

Number Generation

- *Pseudo Random Numbers:* $x_{j+1} = g(x_1, \ldots, x_j)$
  - deterministic & reproducible
  - efficient & transparent
  - almost indistinguishable with long periods of recurrence
- *Equal Distribution Generators:*
  - *Linear Congruential*

Number Generation

- *Pseudo Random Numbers:* $x_{j+1} = g(x_1, \ldots, x_j)$
  - deterministic & reproducible
  - efficient & transparent
  - almost indistinguishable with long periods of recurrence
- *Equal Distribution Generators:*
  - *Linear Congruential*
  - *Linear Feedback Shift*

Number Generation

- *Pseudo Random Numbers:* $x_{j+1} = g(x_1, \dots, x_j)$
  - deterministic & reproducible
  - efficient & transparent
  - almost indistinguishable with long periods of recurrence
- *Equal Distribution Generators:*
  - *Linear Congruential*
  - *Linear Feedback Shift*
  - *Permutated Congruential*

Number Generation

- *Pseudo Random Numbers:* $x_{j+1} = g(x_1, \ldots, x_j)$
    - deterministic & reproducible
    - efficient & transparent
    - almost indistinguishable with long periods of recurrence
- *Equal Distribution Generators:*
    - *Linear Congruential*
    - *Linear Feedback Shift*
    - *Permutated Congruential*
- *Arbitrary Distributions:* obtained from uniformly distributed numbers by various methods

Overview

Motivation

Motivation

- *Model Optimisation:* finding the best possible model parameters

Motivation

- *Model Optimisation:* finding the best possible model parameters
    - maximising objective function *(Fitting)*

Motivation

- *Model Optimisation:* finding the best possible model parameters
  - maximising objective function *(Fitting)*
  - minimising loss function *(Machine Learning)*

Motivation

- *Model Optimisation:* finding the best possible model parameters
  - maximising objective function *(Fitting)*
  - minimising loss function *(Machine Learning)*
- *Example:* modified blackbody spectrum for galactic dust emission (M. Juvela, *A&A* 2023)

Motivation

- *Model Optimisation:* finding the best possible model parameters
  - maximising objective function *(Fitting)*
  - minimising loss function *(Machine Learning)*
- *Example:* modified blackbody spectrum for galactic dust emission (M. Juvela, *A&A* 2023)

$$I_\nu(T,\beta) = B_\nu(T)\left(1 - e^{-\tau_\nu(\beta)}\right) \approx B_\nu(T)\tau_\nu(\beta) = B_\nu(T)\Sigma\kappa_\nu(\beta) = \frac{2h\Sigma\kappa_0}{c^2{\nu_0}^\beta}\frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

Motivation

- *Model Optimisation:* finding the best possible model parameters
  - maximising objective function *(Fitting)*
  - minimising loss function *(Machine Learning)*
- *Example:* modified blackbody spectrum for galactic dust emission (M. Juvela, *A&A* 2023)

$$I_\nu(T,\beta) \equiv \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

Example

Example

$$I_\nu(T, \beta) = \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

Example

$$I_\nu(T,\beta) = \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

- *Objective:* optimise $\theta \equiv (T,\beta)$ such that the weighted squared error is minimal

Example

$$I_\nu(T,\beta) = \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

- *Objective:* optimise $\theta \equiv (T, \beta)$ such that the weighted squared error is minimal
- *Assumptions:*

Example

$$I_\nu(T, \beta) = \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

- *Objective:* optimise $\theta \equiv (T, \beta)$ such that the weighted squared error is minimal
- *Assumptions:*
  - independent measurements generated by $I_\nu(\theta)$ with normally distributed uncertainty

Example

$$I_\nu(T,\beta) = \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

- *Objective:* optimise $\theta \equiv (T, \beta)$ such that the weighted squared error is minimal
- *Assumptions:*
  - independent measurements generated by $I_\nu(\theta)$ with normally distributed uncertainty
  - infinitesimally small error in ν so that we only need σ_ν in I_ν direction

Example

$$I_\nu(T, \beta) = \frac{\nu^{3+\beta}}{\exp\left(\frac{h\nu}{kT}\right) - 1}$$

- *Objective:* optimise $\theta \equiv (T, \beta)$ such that the weighted squared error is minimal
- *Assumptions:*
  - independent measurements generated by $I_\nu(\theta)$ with normally distributed uncertainty
  - infinitesimally small error in ν so that we only need σ_ν in I_ν direction
  - suppose we have no prior information on the parameter distribution

Example

Example

- *Likelihood:*

$$L(I_\nu|\nu,\sigma_\nu,\theta) = \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma_{\nu,i}^2}} \exp\left(-\frac{\left(I_{\nu,i} - I_\nu(\theta)\right)^2}{2\sigma_{\nu,i}^2}\right)$$

Example

- *Likelihood:*

$$\log L(I_\nu|\nu,\sigma_\nu,\theta) = -\frac{N}{2}\log(2\pi) - \sum_{i=1}^{N}\sigma_{\nu,i} - \frac{1}{2}\sum_{i=1}^{N}\frac{\left(I_{\nu,i} - I_\nu(\theta)\right)^2}{\sigma_{\nu,i}^2}$$

Example

- *Likelihood:*

$$-\log L(I_\nu|\nu,\sigma_\nu,\theta) \propto \chi^2(\theta) = \sum_{i=1}^{N} \frac{\left(I_{\nu,i} - I_\nu(\theta)\right)^2}{\sigma_{\nu,i}^2}$$

Example

- *Likelihood:*

$$- \log L(I_\nu | \nu, \sigma_\nu, \theta) \propto \chi^2(\theta) = \sum_{i=1}^{N} \frac{\left(I_{\nu,i} - I_\nu(\theta)\right)^2}{\sigma_{\nu,i}^2}$$

- *Posterior:*

$$P(\theta | I_\nu, \nu, \sigma_\nu) \propto L(I_\nu | \nu, \sigma_\nu, \theta)$$

Example

- *Likelihood:*

$$-\log L(I_\nu|\nu, \sigma_\nu, \theta) \propto \chi^2(\theta) = \sum_{i=1}^{N} \frac{\left(I_{\nu,i} - I_\nu(\theta)\right)^2}{\sigma_{\nu,i}^2}$$

- *Posterior:*

$$\log P(\theta|I_\nu, \nu, \sigma_\nu) \propto \log L(I_\nu|\nu, \sigma_\nu, \theta)$$

Example

- *Likelihood:*

$$-\log L(I_\nu|\nu,\sigma_\nu,\theta) \propto \chi^2(\theta) = \sum_{i=1}^{N} \frac{\left(I_{\nu,i} - I_\nu(\theta)\right)^2}{\sigma_{\nu,i}^2}$$

- *Posterior:*

$$\log P(\theta|I_\nu,\nu,\sigma_\nu) \propto -\chi^2(\theta)$$

Example

- *Likelihood:*

$$-\log L(I_\nu|\nu,\sigma_\nu,\theta) \propto \chi^2(\theta) = \sum_{i=1}^{N} \frac{\left(I_{\nu,i} - I_\nu(\theta)\right)^2}{\sigma_{\nu,i}^2}$$

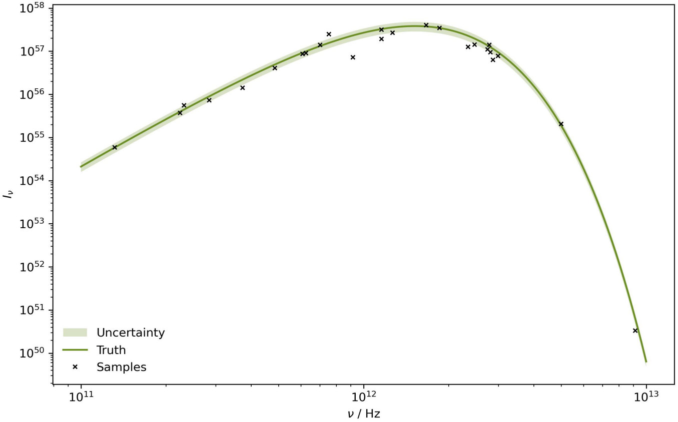
- *Posterior:*

$$\log P(\theta|I_\nu,\nu,\sigma_\nu) \propto -\chi^2(\theta)$$

- *Objective:* minimise the $\chi^2(\theta)$ measure to maximise the posterior probability

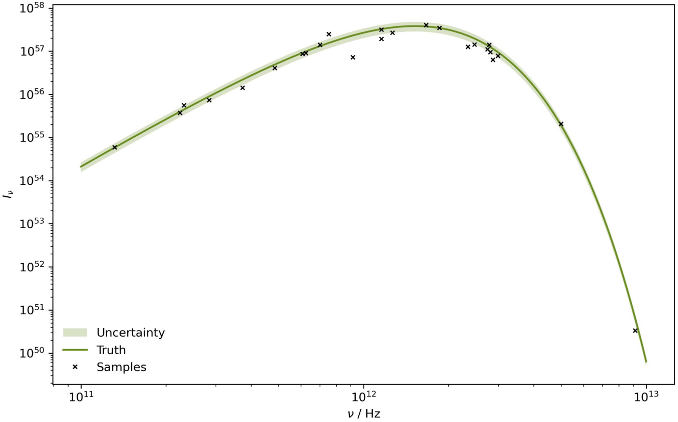
Measurements

Measurements



Measurements

How do we proceed from this dataset to maximise the likelihood for our nonlinear model?



Overview

Parameter Grid

Parameter Grid

- *Idea:* in our $D = 2$ dimensional parameter space, we can create an $N \times N$ grid of (T, β) points and evaluate the scalar function at each one

Parameter Grid

- *Idea:* in our $D = 2$ dimensional parameter space, we can create an $N \times N$ grid of (T, β) points and evaluate the scalar function at each one
- *Preparation:* for this as well as some of the following methods, it is a good idea to normalize the variation of each individual parameter

Parameter Grid

- *Idea:* in our $D = 2$ dimensional parameter space, we can create an $N \times N$ grid of (T, β) points and evaluate the scalar function at each one
- *Preparation:* for this as well as some of the following methods, it is a good idea to normalize the variation of each individual parameter
    - from the literature we expect $(T, \beta) \in [10, 25] \times [1.5, 2.0]$ as constraints

Parameter Grid

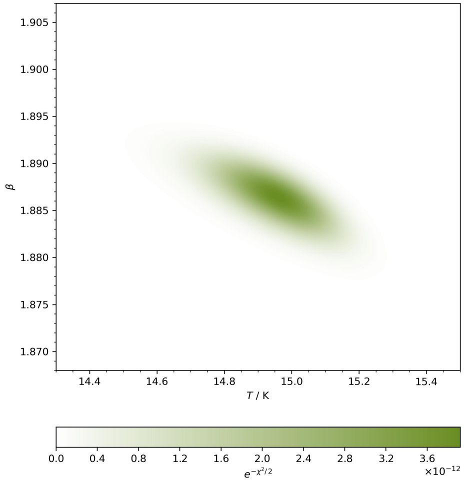
- *Idea:* in our $D = 2$ dimensional parameter space, we can create an $N \times N$ grid of (T, β) points and evaluate the scalar function at each one
- *Preparation:* for this as well as some of the following methods, it is a good idea to normalize the variation of each individual parameter
  - from the literature we expect $(T, \beta) \in [10, 25] \times [1.5, 2.0]$ as constraints
  - resize these intervals by defining $T \mapsto \tilde{T} = T / 30$ with $I_\nu(T, \beta) \to I_\nu(\tilde{T}, \beta)$

Parameter Grid

- *Idea:* in our $D = 2$ dimensional parameter space, we can create an $N \times N$ grid of (T, β) points and evaluate the scalar function at each one
- *Preparation:* for this as well as some of the following methods, it is a good idea to normalize the variation of each individual parameter
  - from the literature we expect $(T, \beta) \in [10, 25] \times [1.5, 2.0]$ as constraints
  - resize these intervals by defining $T \mapsto \tilde{T} = T / 30$ with $I_\nu(T, \beta) \to I_\nu(\tilde{T}, \beta)$
- *Problem:* as D increases we have N^D gridpoints, and with more complex parameter distributions, this procedure quickly becomes intractable due to computational limitations

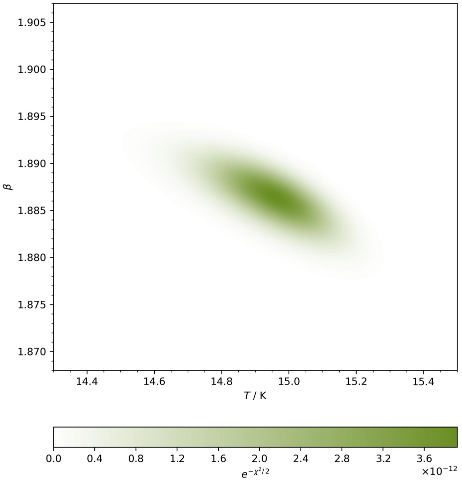
Parameter Grid

Parameter Grid



Parameter Grid

Can we think of more elegant ways to explore even very high dimensional parameter spaces?



General Case

General Case

- *Optimisation Algorithms:* intelligently move through parameter space towards optimum

General Case

- *Optimisation Algorithms:* intelligently move through parameter space towards optimum
- *Common Attributes:*

General Case

- *Optimisation Algorithms:* intelligently move through parameter space towards optimum
- *Common Attributes:*
    1. choose initialisation point

General Case

- *Optimisation Algorithms:* intelligently move through parameter space towards optimum
- *Common Attributes:*
  1. choose initialisation point
  2. evaluate objective function or its derivatives to estimate local curvature

General Case

- *Optimisation Algorithms:* intelligently move through parameter space towards optimum
- *Common Attributes:*
  1. choose initialisation point
  2. evaluate objective function or its derivatives to estimate local curvature
  3. propose new step in parameter space

General Case

- *Optimisation Algorithms:* intelligently move through parameter space towards optimum
- *Common Attributes:*
  1. choose initialisation point
  2. evaluate objective function or its derivatives to estimate local curvature
  3. propose new step in parameter space
  4. decide whether proposal is a good step and update position accordingly

General Case

- *Optimisation Algorithms:* intelligently move through parameter space towards optimum
- *Common Attributes:*
  1. choose initialisation point
  2. evaluate objective function or its derivatives to estimate local curvature
  3. propose new step in parameter space
  4. decide whether proposal is a good step and update position accordingly
  5. repeat until some predefined cutoff is satisfied

General Case

- *Optimisation Algorithms:* intelligently move through parameter space towards optimum
- *Common Attributes:*
  1. choose initialisation point
  2. evaluate objective function or its derivatives to estimate local curvature
  3. propose new step in parameter space
  4. decide whether proposal is a good step and update position accordingly
  5. repeat until some predefined cutoff is satisfied
- *Interactive Visualisations:* check out Ben Frederickson[1] and Andy Casey[2] to see some nice examples

Nelder-Mead

Nelder-Mead

- *Requirements:* only objective function, no information on gradients

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
    1. initialize simplex of $d + 1$ points in d dimensional parameter space

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. initialize simplex of $d+1$ points in d dimensional parameter space
  2. rank points in order of values $f(x_1) \leq \cdots \leq f(x_d) \leq f(x_{d+1})$

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. initialize simplex of $d + 1$ points in d dimensional parameter space
  2. rank points in order of values $f(x_1) \leq \cdots \leq f(x_d) \leq f(x_{d+1})$
  3. compute centroid x_0 of $x_1, \ldots, x_d$

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. initialize simplex of $d + 1$ points in d dimensional parameter space
  2. rank points in order of values $f(x_1) \leq \cdots \leq f(x_d) \leq f(x_{d+1})$
  3. compute centroid x_0 of $x_1, \ldots, x_d$
  4. perform a bunch of conditional evaluations between x_0 and x_{d+1}

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. initialize simplex of $d + 1$ points in d dimensional parameter space
  2. rank points in order of values $f(x_1) \leq \cdots \leq f(x_d) \leq f(x_{d+1})$
  3. compute centroid x_0 of $x_1, \ldots, x_d$
  4. perform a bunch of conditional evaluations between x_0 and x_{d+1}
  5. reflect, expand, contract or shrink based on results to move points downhill and repeat

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. initialize simplex of $d+1$ points in d dimensional parameter space
  2. rank points in order of values $f(x_1) \leq \cdots \leq f(x_d) \leq f(x_{d+1})$
  3. compute centroid x_0 of $x_1, \ldots, x_d$
  4. perform a bunch of conditional evaluations between x_0 and x_{d+1}
  5. reflect, expand, contract or shrink based on results to move points downhill and repeat
- *Notes:*

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. initialize simplex of $d + 1$ points in d dimensional parameter space
  2. rank points in order of values $f(x_1) \leq \cdots \leq f(x_d) \leq f(x_{d+1})$
  3. compute centroid x_0 of $x_1, \ldots, x_d$
  4. perform a bunch of conditional evaluations between x_0 and x_{d+1}
  5. reflect, expand, contract or shrink based on results to move points downhill and repeat
- *Notes:*
  - extrapolates function to slowly move geometric blob through parameter space

Nelder-Mead

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. initialize simplex of $d + 1$ points in d dimensional parameter space
  2. rank points in order of values $f(x_1) \leq \cdots \leq f(x_d) \leq f(x_{d+1})$
  3. compute centroid x_0 of $x_1, \ldots, x_d$
  4. perform a bunch of conditional evaluations between x_0 and x_{d+1}
  5. reflect, expand, contract or shrink based on results to move points downhill and repeat
- *Notes:*
  - extrapolates function to slowly move geometric blob through parameter space
  - zeroth order method since no derivatives are used

Nelder-Mead

Nelder-Mead

- *Conditions:*

Nelder-Mead

- *Conditions:*
  1. *Reflection:*

Nelder-Mead

- *Conditions:*
  1. *Reflection:*
     - compute reflected point $x_r = x_0 + \alpha(x_0 - x_{d+1})$ with $\alpha > 0$

Nelder-Mead

- *Conditions:*
  1. *Reflection:*
     - compute reflected point $x_r = x_0 + \alpha(x_0 - x_{d+1})$ with $\alpha > 0$
     - if x_r better than second worst x_d but not better than best x_1 with $f(x_1) \leq f(x_r) < f(x_d)$ then obtain new simplex by replacing x_{d+1} with x_r and repeat from start

Nelder-Mead

- *Conditions:*
  1. *Reflection:*
     - compute reflected point $x_r = x_0 + \alpha(x_0 - x_{d+1})$ with $\alpha > 0$
     - if x_r better than second worst x_d but not better than best x_1 with $f(x_1) \leq f(x_r) < f(x_d)$ then obtain new simplex by replacing x_{d+1} with x_r and repeat from start
     - continue to the next step instead if x_r is the best point with $f(x_r) < f(x_1)$

Nelder-Mead

- *Conditions:*
  1. *Reflection:*
     - compute reflected point $x_r = x_0 + \alpha(x_0 - x_{d+1})$ with $\alpha > 0$
     - if x_r better than second worst x_d but not better than best x_1 with $f(x_1) \leq f(x_r) < f(x_d)$ then obtain new simplex by replacing x_{d+1} with x_r and repeat from start
     - continue to the next step instead if x_r is the best point with $f(x_r) < f(x_1)$
     - skip the next skip and go to the one after it if x_r is not better than the second worst x_d with $f(x_r) \geq f(x_d)$

Nelder-Mead

Nelder-Mead

- *Conditions:*

Nelder-Mead

- *Conditions:*
  2. *Expansion:*

Nelder-Mead

- *Conditions:*
  2. *Expansion:*
     - compute expanded point $x_e = x_0 + \gamma(x_r - x_0)$ with $\gamma > 1$

Nelder-Mead

- *Conditions:*
  2. *Expansion:*
     - compute expanded point $x_e = x_0 + \gamma(x_r - x_0)$ with $\gamma > 1$
     - if x_e is better than x_r with $f(x_e) < f(x_r)$ then obtain new simplex by replacing the worst x_{d+1} with x_e and repeat from start

Nelder-Mead

- *Conditions:*
  2. *Expansion:*
     - compute expanded point $x_e = x_0 + \gamma(x_r - x_0)$ with $\gamma > 1$
     - if x_e is better than x_r with $f(x_e) < f(x_r)$ then obtain new simplex by replacing the worst x_{d+1} with x_e and repeat from start
     - else obtain new simplex by replacing the worst x_{d+1} with x_r and repeat from start

Nelder-Mead

- *Conditions:*
  2. *Expansion:*
     - compute expanded point $x_e = x_0 + \gamma(x_r - x_0)$ with $\gamma > 1$
     - if x_e is better than x_r with $f(x_e) < f(x_r)$ then obtain new simplex by replacing the worst x_{d+1} with x_e and repeat from start
     - else obtain new simplex by replacing the worst x_{d+1} with x_r and repeat from start
  3. *Contraction:*

Nelder-Mead

- *Conditions:*
  2. *Expansion:*
     - compute expanded point $x_e = x_0 + \gamma(x_r - x_0)$ with $\gamma > 1$
     - if x_e is better than x_r with $f(x_e) < f(x_r)$ then obtain new simplex by replacing the worst x_{d+1} with x_e and repeat from start
     - else obtain new simplex by replacing the worst x_{d+1} with x_r and repeat from start
  3. *Contraction:*
     - if x_r is better than the worst x_{d+1} with $f(x_r) < f(x_{d+1})$ then compute the outside contracted point $x_c = x_0 + \rho(x_r - x_0)$ where $0 < \rho \leq 0.5$

Nelder-Mead

- *Conditions:*
  2. *Expansion:*
     - compute expanded point $x_e = x_0 + \gamma(x_r - x_0)$ with $\gamma > 1$
     - if x_e is better than x_r with $f(x_e) < f(x_r)$ then obtain new simplex by replacing the worst x_{d+1} with x_e and repeat from start
     - else obtain new simplex by replacing the worst x_{d+1} with x_r and repeat from start
  3. *Contraction:*
     - if x_r is better than the worst x_{d+1} with $f(x_r) < f(x_{d+1})$ then compute the outside contracted point $x_c = x_0 + \rho(x_r - x_0)$ where $0 < \rho \leq 0.5$
     - for x_c better than x_r with $f(x_c) < f(x_r)$ obtain new simplex by replacing worst x_{d+1} with x_c and repeat from start, otherwise go to the next step

Nelder-Mead

Nelder-Mead

- *Conditions:*

Nelder-Mead

- *Conditions:*
  3. *Contraction:*

Nelder-Mead

- *Conditions:*
  3. *Contraction:*
     - if x_r is not better than the worst x_{d+1} with $f(x_r) \geq f(x_{d+1})$ then compute the inside contracted point $x_c = x_0 + \rho(x_{d+1} - x_0)$ where $0 < \rho \leq 0.5$

Nelder-Mead

- *Conditions:*
  3. *Contraction:*
     - if x_r is not better than the worst x_{d+1} with $f(x_r) \geq f(x_{d+1})$ then compute the inside contracted point $x_c = x_0 + \rho(x_{d+1} - x_0)$ where $0 < \rho \leq 0.5$
     - for x_c better than the worst x_{d+1} with $f(x_c) < f(x_{d+1})$ obtain new simplex by replacing worst x_{d+1} with x_c and repeat from start, otherwise go to next step

Nelder-Mead

- *Conditions:*
  3. *Contraction:*
     - if x_r is not better than the worst x_{d+1} with $f(x_r) \geq f(x_{d+1})$ then compute the inside contracted point $x_c = x_0 + \rho(x_{d+1} - x_0)$ where $0 < \rho \leq 0.5$
     - for x_c better than the worst x_{d+1} with $f(x_c) < f(x_{d+1})$ obtain new simplex by replacing worst x_{d+1} with x_c and repeat from start, otherwise go to next step
  4. *Shrink:*

Nelder-Mead

- *Conditions:*
  3. *Contraction:*
     - if x_r is not better than the worst x_{d+1} with $f(x_r) \geq f(x_{d+1})$ then compute the inside contracted point $x_c = x_0 + \rho(x_{d+1} - x_0)$ where $0 < \rho \leq 0.5$
     - for x_c better than the worst x_{d+1} with $f(x_c) < f(x_{d+1})$ obtain new simplex by replacing worst x_{d+1} with x_c and repeat from start, otherwise go to next step
  4. *Shrink:*
     - obtain new simplex by replacing all points except the best x_1 with $x_i = x_1 + \sigma(x_i - x_1)$ and coefficient $0 < \sigma < 1$ before repeating from start

Nelder-Mead

- *Conditions:*
  3. *Contraction:*
     - if x_r is not better than the worst x_{d+1} with $f(x_r) \geq f(x_{d+1})$ then compute the inside contracted point $x_c = x_0 + \rho(x_{d+1} - x_0)$ where $0 < \rho \leq 0.5$
     - for x_c better than the worst x_{d+1} with $f(x_c) < f(x_{d+1})$ obtain new simplex by replacing worst x_{d+1} with x_c and repeat from start, otherwise go to next step
  4. *Shrink:*
     - obtain new simplex by replacing all points except the best x_1 with $x_i = x_1 + \sigma(x_i - x_1)$ and coefficient $0 < \sigma < 1$ before repeating from start
- *Standard Values:* $\alpha = 1$ & $\gamma = 2$ & $\rho = \sigma = 0.5$

Random Descent

Random Descent

- *Requirements:* only objective function, no information on gradients

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. start from arbitrary inital value x_0

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. start from arbitrary inital value x_0
  2. draw a proposed shift from some chosen probability distribution $X \sim g(x)$

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. start from arbitrary inital value x_0
  2. draw a proposed shift from some chosen probability distribution $X \sim g(x)$
  3. propose updated position $\hat{x}_{k+1} = x_k + X$

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. start from arbitrary inital value x_0
  2. draw a proposed shift from some chosen probability distribution $X \sim g(x)$
  3. propose updated position $\hat{x}_{k+1} = x_k + X$
  4. accept proposal $x_{k+1} = \hat{x}_{k+1}$ only if $f(\hat{x}_{k+1}) < f(x_{k+1})$ and repeat

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. start from arbitrary inital value x_0
  2. draw a proposed shift from some chosen probability distribution $X \sim g(x)$
  3. propose updated position $\hat{x}_{k+1} = x_k + X$
  4. accept proposal $x_{k+1} = \hat{x}_{k+1}$ only if $f(\hat{x}_{k+1}) < f(x_{k+1})$ and repeat
- *Notes:*

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. start from arbitrary inital value x_0
  2. draw a proposed shift from some chosen probability distribution $X \sim g(x)$
  3. propose updated position $\hat{x}_{k+1} = x_k + X$
  4. accept proposal $x_{k+1} = \hat{x}_{k+1}$ only if $f(\hat{x}_{k+1}) < f(x_{k+1})$ and repeat
- *Notes:*
  - traverses parameter space via modified random walk, can be very slow

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. start from arbitrary inital value x_0
  2. draw a proposed shift from some chosen probability distribution $X \sim g(x)$
  3. propose updated position $\hat{x}_{k+1} = x_k + X$
  4. accept proposal $x_{k+1} = \hat{x}_{k+1}$ only if $f(\hat{x}_{k+1}) < f(x_{k+1})$ and repeat
- *Notes:*
  - traverses parameter space via modified random walk, can be very slow
  - zeroth order method since no derivatives are used

Random Descent

- *Requirements:* only objective function, no information on gradients
- *Algorithm:*
  1. start from arbitrary inital value x_0
  2. draw a proposed shift from some chosen probability distribution $X \sim g(x)$
  3. propose updated position $\hat{x}_{k+1} = x_k + X$
  4. accept proposal $x_{k+1} = \hat{x}_{k+1}$ only if $f(\hat{x}_{k+1}) < f(x_{k+1})$ and repeat
- *Notes:*
  - traverses parameter space via modified random walk, can be very slow
  - zeroth order method since no derivatives are used
  - improve by considering last step and optimizing step range

Gradient Descent

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
    1. initialize at some x_0

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize at some x_0
  2. update current point with $x_{k+1} = x_k - \alpha \nabla f(x_k)$

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
    1. initialize at some x_0
    2. update current point with $x_{k+1} = x_k - \alpha \nabla f(x_k)$
    3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize at some x_0
  2. update current point with $x_{k+1} = x_k - \alpha \nabla f(x_k)$
  3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize at some x_0
  2. update current point with $x_{k+1} = x_k - \alpha \nabla f(x_k)$
  3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*
  - moves along direction of steepest descent at each point with fixed learning rate α

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize at some x_0
  2. update current point with $x_{k+1} = x_k - \alpha \nabla f(x_k)$
  3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*
  - moves along direction of steepest descent at each point with fixed learning rate α
  - first order method since only the gradient is used

Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize at some x_0
  2. update current point with $x_{k+1} = x_k - \alpha \nabla f(x_k)$
  3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*
  - moves along direction of steepest descent at each point with fixed learning rate α
  - first order method since only the gradient is used
  - improve by performing line search for α at every step

Backtracking Line Search

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
    1. start from some fixed large α_0 for each step k of the gradient descent

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. start from some fixed large α_0 for each step k of the gradient descent
  2. set $\alpha_{j+1} = \tau\alpha_j$ until $f\left(x_k\right) - f\left(x_k - \alpha_j \nabla f(x_k)\right) \geq \kappa\alpha_j \left(\nabla f(x_k)\right)^2$ where $\tau, \kappa \in (0, 1)$

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. start from some fixed large α_0 for each step k of the gradient descent
  2. set $\alpha_{j+1} = \tau\alpha_j$ until $f\left(x_k\right) - f\left(x_k - \alpha_j \nabla f(x_k)\right) \geq \kappa\alpha_j \left(\nabla f(x_k)\right)^2$ where $\tau, \kappa \in (0, 1)$
  3. use last value to set learning rate $\alpha = \alpha_j$ for next step

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
    1. start from some fixed large $\boldsymbol{\alpha_0}$ for each step $\boldsymbol{k}$ of the gradient descent
    2. set $\alpha_{j+1} = \tau\alpha_j$ until $f\left(x_k\right) - f\left(x_k - \alpha_j \nabla f(x_k)\right) \geq \kappa\alpha_j \left(\nabla f(x_k)\right)^2$ where $\boldsymbol{\tau, \kappa \in (0, 1)}$
    3. use last value to set learning rate $\boldsymbol{\alpha = \alpha_j}$ for next step
- *Notes:*

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. start from some fixed large $\boldsymbol{\alpha_0}$ for each step $\boldsymbol{k}$ of the gradient descent
  2. set $\alpha_{j+1} = \tau\alpha_j$ until $f(x_k) - f\left(x_k - \alpha_j \nabla f(x_k)\right) \geq \kappa\alpha_j \left(\nabla f(x_k)\right)^2$ where $\boldsymbol{\tau, \kappa \in (0, 1)}$
  3. use last value to set learning rate $\boldsymbol{\alpha = \alpha_j}$ for next step
- *Notes:*
  - ensures $f(x_{k+1}) < f(x_k)$ such that the function steadily descreases with increasing $\boldsymbol{k}$

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. start from some fixed large α_0 for each step k of the gradient descent
  2. set $\alpha_{j+1} = \tau\alpha_j$ until $f\left(x_k\right) - f\left(x_k - \alpha_j \nabla f(x_k)\right) \geq \kappa\alpha_j \left(\nabla f(x_k)\right)^2$ where $\tau, \kappa \in (0, 1)$
  3. use last value to set learning rate $\alpha = \alpha_j$ for next step
- *Notes:*
  - ensures $f(x_{k+1}) < f(x_k)$ such that the function steadily descreases with increasing k
  - favours secant between x_k and x_{k+1} steeper or almost as steep as tangent $\nabla f(x_k)$

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. start from some fixed large α_0 for each step k of the gradient descent
  2. set $\alpha_{j+1} = \tau\alpha_j$ until $f(x_k) - f\left(x_k - \alpha_j \nabla f(x_k)\right) \geq \kappa\alpha_j \left(\nabla f(x_k)\right)^2$ where $\tau, \kappa \in (0, 1)$
  3. use last value to set learning rate $\alpha = \alpha_j$ for next step
- *Notes:*
  - ensures $f(x_{k+1}) < f(x_k)$ such that the function steadily descreases with increasing k
  - favours secant between x_k and x_{k+1} steeper or almost as steep as tangent $\nabla f(x_k)$
  - robust convergence guarantees, larger step size for flat areas avoids saddle points

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. start from some fixed large α_0 for each step k of the gradient descent
  2. set $\alpha_{j+1} = \tau\alpha_j$ until $f(x_k) - f\left(x_k - \alpha_j \nabla f(x_k)\right) \geq \kappa\alpha_j \left(\nabla f(x_k)\right)^2$ where $\tau, \kappa \in (0, 1)$
  3. use last value to set learning rate $\alpha = \alpha_j$ for next step
- *Notes:*
  - ensures $f(x_{k+1}) < f(x_k)$ such that the function steadily descreases with increasing k
  - favours secant between x_k and x_{k+1} steeper or almost as steep as tangent $\nabla f(x_k)$
  - robust convergence guarantees, larger step size for flat areas avoids saddle points
  - first order method since only the gradient is used

Backtracking Line Search

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. start from some fixed large α_0 for each step k of the gradient descent
  2. set $\alpha_{j+1} = \tau\alpha_j$ until $f(x_k) - f\left(x_k - \alpha_j \nabla f(x_k)\right) \geq \kappa\alpha_j \left(\nabla f(x_k)\right)^2$ where $\tau, \kappa \in (0, 1)$
  3. use last value to set learning rate $\alpha = \alpha_j$ for next step
- *Notes:*
  - ensures $f(x_{k+1}) < f(x_k)$ such that the function steadily descreases with increasing k
  - favours secant between x_k and x_{k+1} steeper or almost as steep as tangent $\nabla f(x_k)$
  - robust convergence guarantees, larger step size for flat areas avoids saddle points
  - first order method since only the gradient is used
  - typical settings are $\tau = \kappa = 0.5$ (L. Armijo, *Pac. J. Math.* 1966)

Conjugate Gradient Descent

Conjugate Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically

Conjugate Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*

Conjugate Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. calculate search direction $s_k = \nabla f(x_k) + \beta s_{k-1}$ where $\beta = \dfrac{\nabla f(x_k)\left(\nabla f(x_k) - \nabla f(x_{k-1})\right)}{\left(\nabla f(x_{k-1})\right)^2}$

Conjugate Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. calculate search direction $s_k = \nabla f(x_k) + \beta s_{k-1}$ where $\beta = \dfrac{\nabla f(x_k)\left(\nabla f(x_k) - \nabla f(x_{k-1})\right)}{\left(\nabla f(x_{k-1})\right)^2}$
  2. update point with $x_{k+1} = x_k + \alpha s_k$ after a line search for α

Conjugate Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. calculate search direction $s_k = \nabla f(x_k) + \beta s_{k-1}$ where $\beta = \dfrac{\nabla f(x_k)\left(\nabla f(x_k) - \nabla f(x_{k-1})\right)}{\left(\nabla f(x_{k-1})\right)^2}$
  2. update point with $x_{k+1} = x_k + \alpha s_k$ after a line search for α
  3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached

Conjugate Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. calculate search direction $s_k = \nabla f(x_k) + \beta s_{k-1}$ where $\beta = \dfrac{\nabla f(x_k)\left(\nabla f(x_k) - \nabla f(x_{k-1})\right)}{\left(\nabla f(x_{k-1})\right)^2}$
  2. update point with $x_{k+1} = x_k + \alpha s_k$ after a line search for α
  3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*

Conjugate Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. calculate search direction $s_k = \nabla f(x_k) + \beta s_{k-1}$ where $\beta = \dfrac{\nabla f(x_k)\left(\nabla f(x_k) - \nabla f(x_{k-1})\right)}{\left(\nabla f(x_{k-1})\right)^2}$
  2. update point with $x_{k+1} = x_k + \alpha s_k$ after a line search for α
  3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*
  - uses gradients from previous evaluations to estimate curvature

Conjugate Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. calculate search direction $s_k = \nabla f(x_k) + \beta s_{k-1}$ where $\beta = \dfrac{\nabla f(x_k)\left(\nabla f(x_k) - \nabla f(x_{k-1})\right)}{\left(\nabla f(x_{k-1})\right)^2}$
  2. update point with $x_{k+1} = x_k + \alpha s_k$ after a line search for α
  3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*
  - uses gradients from previous evaluations to estimate curvature
  - smooths oscillatory behaviour of naive gradient descent, especially in tight valleys with nearby gradients almost orthogonal to search direction

Conjugate Gradient Descent

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. calculate search direction $s_k = \nabla f(x_k) + \beta s_{k-1}$ where $\beta = \dfrac{\nabla f(x_k)\left(\nabla f(x_k) - \nabla f(x_{k-1})\right)}{\left(\nabla f(x_{k-1})\right)^2}$
  2. update point with $x_{k+1} = x_k + \alpha s_k$ after a line search for α
  3. iterate until convergence $|x_{k+1}/x_k - 1| < \epsilon$ or maximum step count is reached
- *Notes:*
  - uses gradients from previous evaluations to estimate curvature
  - smooths oscillatory behaviour of naive gradient descent, especially in tight valleys with nearby gradients almost orthogonal to search direction
  - first order method since only the gradient is used

Adaptive Moment Estimation

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
    1. initialize position x_0 as well as moving averages $m_0 = 0$ and $v_0 = 0$

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize position x_0 as well as moving averages $m_0 = 0$ and $v_0 = 0$
  2. set $m_{k+1} = \beta_1 m_k + (1 - \beta_1)\nabla f(x_k)$ and $v_{k+1} = \beta_2 v_k + (1 - \beta_2)\left(\nabla f(x_k)\right)^2$

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize position x_0 as well as moving averages $m_0 = 0$ and $v_0 = 0$
  2. set $m_{k+1} = \beta_1 m_k + (1 - \beta_1)\nabla f(x_k)$ and $v_{k+1} = \beta_2 v_k + (1 - \beta_2)\left(\nabla f(x_k)\right)^2$
  3. correct estimator bias with $\tilde{m}_k = m_{k+1}(1 - \beta_1^k)^{-1}$ and $\tilde{v}_k = v_{k+1}(1 - \beta_2^k)^{-1}$

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize position x_0 as well as moving averages $m_0 = 0$ and $v_0 = 0$
  2. set $m_{k+1} = \beta_1 m_k + (1 - \beta_1)\nabla f(x_k)$ and $v_{k+1} = \beta_2 v_k + (1 - \beta_2)\left(\nabla f(x_k)\right)^2$
  3. correct estimator bias with $\tilde{m}_k = m_{k+1}(1 - \beta_1^k)^{-1}$ and $\tilde{v}_k = v_{k+1}(1 - \beta_2^k)^{-1}$
  4. update the position $x_{k+1} = x_k - \alpha \tilde{m}_k \left(\sqrt{\tilde{v}_k} + \varepsilon\right)^{-1}$ and repeat

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize position x_0 as well as moving averages $m_0 = 0$ and $v_0 = 0$
  2. set $m_{k+1} = \beta_1 m_k + (1 - \beta_1)\nabla f(x_k)$ and $v_{k+1} = \beta_2 v_k + (1 - \beta_2)\left(\nabla f(x_k)\right)^2$
  3. correct estimator bias with $\tilde{m}_k = m_{k+1}(1 - \beta_1^k)^{-1}$ and $\tilde{v}_k = v_{k+1}(1 - \beta_2^k)^{-1}$
  4. update the position $x_{k+1} = x_k - \alpha\tilde{m}_k\left(\sqrt{\tilde{v}_k} + \varepsilon\right)^{-1}$ and repeat
- *Notes:*

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize position x_0 as well as moving averages $m_0 = 0$ and $v_0 = 0$
  2. set $m_{k+1} = \beta_1 m_k + (1 - \beta_1)\nabla f(x_k)$ and $v_{k+1} = \beta_2 v_k + (1 - \beta_2)\left(\nabla f(x_k)\right)^2$
  3. correct estimator bias with $\tilde{m}_k = m_{k+1}(1 - \beta_1^k)^{-1}$ and $\tilde{v}_k = v_{k+1}(1 - \beta_2^k)^{-1}$
  4. update the position $x_{k+1} = x_k - \alpha \tilde{m}_k \left(\sqrt{\tilde{v}_k} + \varepsilon\right)^{-1}$ and repeat
- *Notes:*
  - point behaves as iterative analogue to ball rolling in landscape with friction

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize position x_0 as well as moving averages $m_0 = 0$ and $v_0 = 0$
  2. set $m_{k+1} = \beta_1 m_k + (1 - \beta_1)\nabla f(x_k)$ and $v_{k+1} = \beta_2 v_k + (1 - \beta_2)\big(\nabla f(x_k)\big)^2$
  3. correct estimator bias with $\tilde{m}_k = m_{k+1}(1 - \beta_1^k)^{-1}$ and $\tilde{v}_k = v_{k+1}(1 - \beta_2^k)^{-1}$
  4. update the position $x_{k+1} = x_k - \alpha \tilde{m}_k \left(\sqrt{\tilde{v}_k} + \varepsilon\right)^{-1}$ and repeat
- *Notes:*
  - point behaves as iterative analogue to ball rolling in landscape with friction
  - hyperparameters $\beta_1, \beta_2 \in (0, 1)$ are forgetting factors for previous gradients, and $\varepsilon \ll 1$

Adaptive Moment Estimation

- *Requirements:* first derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. initialize position x_0 as well as moving averages $m_0 = 0$ and $v_0 = 0$
  2. set $m_{k+1} = \beta_1 m_k + (1 - \beta_1)\nabla f(x_k)$ and $v_{k+1} = \beta_2 v_k + (1 - \beta_2)\left(\nabla f(x_k)\right)^2$
  3. correct estimator bias with $\tilde{m}_k = m_{k+1}(1 - \beta_1^k)^{-1}$ and $\tilde{v}_k = v_{k+1}(1 - \beta_2^k)^{-1}$
  4. update the position $x_{k+1} = x_k - \alpha\tilde{m}_k\left(\sqrt{\tilde{v}_k} + \varepsilon\right)^{-1}$ and repeat
- *Notes:*
  - point behaves as iterative analogue to ball rolling in landscape with friction
  - hyperparameters $\beta_1, \beta_2 \in (0, 1)$ are forgetting factors for previous gradients, and $\varepsilon \ll 1$
  - first order method since only the gradient is used

Newton Method

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically
- *Algorithm:*

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically
- *Algorithm:*
    1. choose some initial x_0

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. choose some initial x_0
  2. calculate $\nabla f(x_k) = \left(\frac{\partial f(x_k)}{\partial z_i}\right)_{i=1,\dots,d}$ and $H_{f(x_k)} = \left(\frac{\partial^2 f(x_k)}{\partial z_i \partial z_j}\right)_{i,j=1,\dots,d}$

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. choose some initial x_0
  2. calculate $\nabla f(x_k) = \left(\frac{\partial f(x_k)}{\partial z_i}\right)_{i=1,\dots,d}$ and $H_{f(x_k)} = \left(\frac{\partial^2 f(x_k)}{\partial z_i \partial z_j}\right)_{i,j=1,\dots,d}$
  3. update the parameters $x_{k+1} = x_k - \alpha H^{-1}_{f(x_k)} \nabla f(x_k)$ with dampening α

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. choose some initial x_0
  2. calculate $\nabla f(x_k) = \left(\frac{\partial f(x_k)}{\partial z_i}\right)_{i=1,\ldots,d}$ and $H_{f(x_k)} = \left(\frac{\partial^2 f(x_k)}{\partial z_i \partial z_j}\right)_{i,j=1,\ldots,d}$
  3. update the parameters $x_{k+1} = x_k - \alpha H^{-1}_{f(x_k)} \nabla f(x_k)$ with dampening α
- *Notes:*

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. choose some initial x_0
  2. calculate $\nabla f(x_k) = \left(\frac{\partial f(x_k)}{\partial z_i}\right)_{i=1,\ldots,d}$ and $H_{f(x_k)} = \left(\frac{\partial^2 f(x_k)}{\partial z_i \partial z_j}\right)_{i,j=1,\ldots,d}$
  3. update the parameters $x_{k+1} = x_k - \alpha H^{-1}_{f(x_k)} \nabla f(x_k)$ with dampening α
- *Notes:*
  - generalises Newton-Raphson method for finding roots of derivative to multiple dimensions

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. choose some initial x_0
  2. calculate $\nabla f(x_k) = \left(\frac{\partial f(x_k)}{\partial z_i}\right)_{i=1,\dots,d}$ and $H_{f(x_k)} = \left(\frac{\partial^2 f(x_k)}{\partial z_i \partial z_j}\right)_{i,j=1,\dots,d}$
  3. update the parameters $x_{k+1} = x_k - \alpha H^{-1}_{f(x_k)} \nabla f(x_k)$ with dampening α
- *Notes:*
  - generalises Newton-Raphson method for finding roots of derivative to multiple dimensions
  - finds critical point, no matter if maximum, minimum or saddle point, sensitive to initialisation

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. choose some initial x_0
  2. calculate $\nabla f(x_k) = \left(\frac{\partial f(x_k)}{\partial z_i}\right)_{i=1,\ldots,d}$ and $H_{f(x_k)} = \left(\frac{\partial^2 f(x_k)}{\partial z_i \partial z_j}\right)_{i,j=1,\ldots,d}$
  3. update the parameters $x_{k+1} = x_k - \alpha H^{-1}_{f(x_k)} \nabla f(x_k)$ with dampening α
- *Notes:*
  - generalises Newton-Raphson method for finding roots of derivative to multiple dimensions
  - finds critical point, no matter if maximum, minimum or saddle point, sensitive to initialisation
  - second order method since both gradient and Hessian are used

Newton Method

- *Requirements:* first and second derivatives of the objective function, can be estimated numerically
- *Algorithm:*
  1. choose some initial x_0
  2. calculate $\nabla f(x_k) = \left(\frac{\partial f(x_k)}{\partial z_i}\right)_{i=1,\ldots,d}$ and $H_{f(x_k)} = \left(\frac{\partial^2 f(x_k)}{\partial z_i \partial z_j}\right)_{i,j=1,\ldots,d}$
  3. update the parameters $x_{k+1} = x_k - \alpha H^{-1}_{f(x_k)} \nabla f(x_k)$ with dampening α
- *Notes:*
  - generalises Newton-Raphson method for finding roots of derivative to multiple dimensions
  - finds critical point, no matter if maximum, minimum or saddle point, sensitive to initialisation
  - second order method since both gradient and Hessian are used
  - related Gauss-Newton algorithm employs Jacobian instead of Hessian

Advanced Algorithms

Advanced Algorithms

- *Traits:* quasi second order methods, interpolate between optimsers, various curvature estimations

Advanced Algorithms

- *Traits:* quasi second order methods, interpolate between optimsers, various curvature estimations
  - Broyden–Fletcher–Goldfarb–Shanno

Advanced Algorithms

- *Traits:* quasi second order methods, interpolate between optimsers, various curvature estimations
  - Broyden–Fletcher–Goldfarb–Shanno
  - Sequential Least Squares Programming

Advanced Algorithms

- *Traits:* quasi second order methods, interpolate between optimsers, various curvature estimations
  - Broyden–Fletcher–Goldfarb–Shanno
  - Sequential Least Squares Programming
  - Levenberg–Marquardt

Advanced Algorithms

- *Traits:* quasi second order methods, interpolate between optimsers, various curvature estimations
    - Broyden–Fletcher–Goldfarb–Shanno
    - Sequential Least Squares Programming
    - Levenberg–Marquardt
    - Migrad

Advanced Algorithms

- *Traits:* quasi second order methods, interpolate between optimsers, various curvature estimations
  - Broyden–Fletcher–Goldfarb–Shanno
  - Sequential Least Squares Programming
  - Levenberg–Marquardt
  - Migrad
  - Root Mean Square Propagation

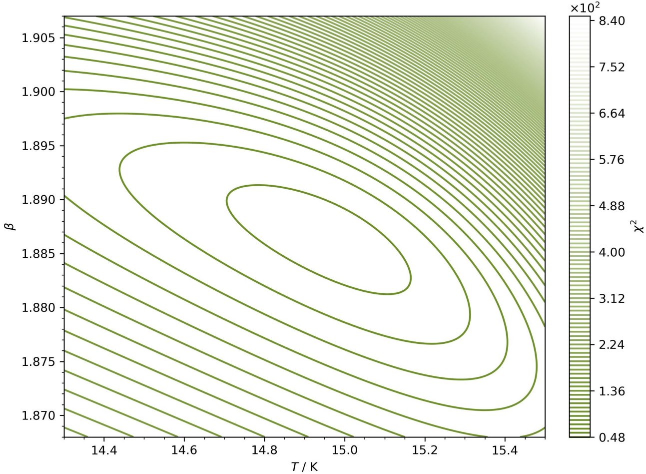
Advanced Algorithms

- *Traits:* quasi second order methods, interpolate between optimsers, various curvature estimations
  - Broyden–Fletcher–Goldfarb–Shanno
  - Sequential Least Squares Programming
  - Levenberg–Marquardt
  - Migrad
  - Root Mean Square Propagation
- *Usage:* found in many modern libraries, suited to specialised use cases like nonlinear least squares, problems with parameter bounds or arbitrary constraints

Application

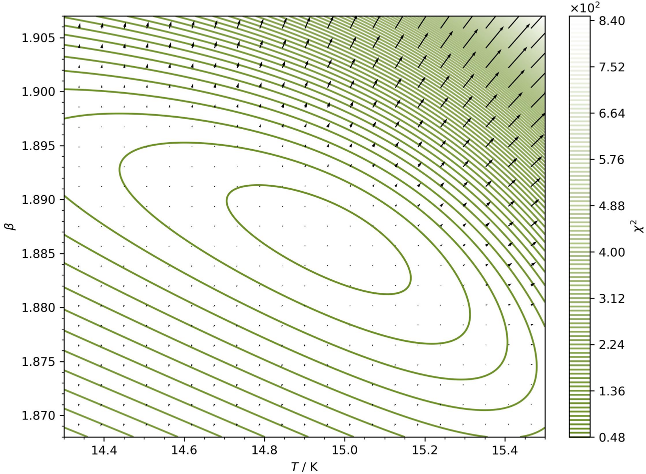
Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm



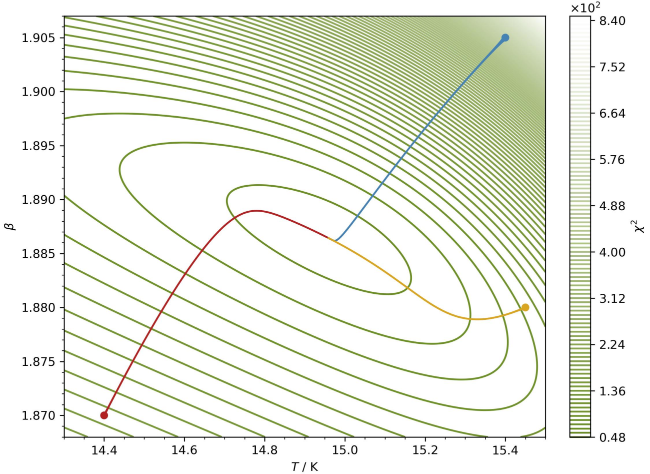
Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm



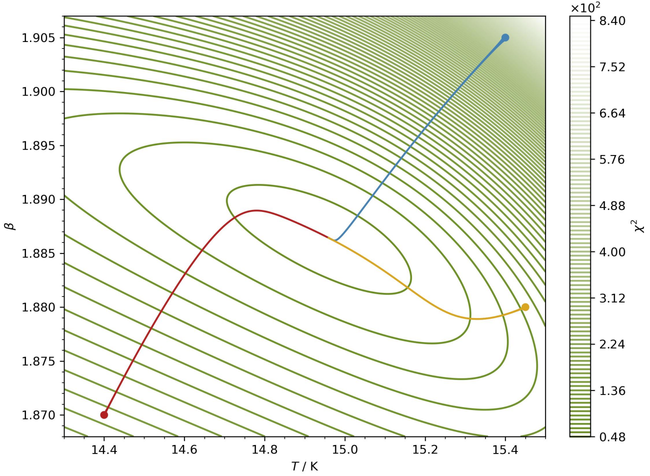
Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm
- *Fixed Rate:* $\alpha = 10^{-9}$



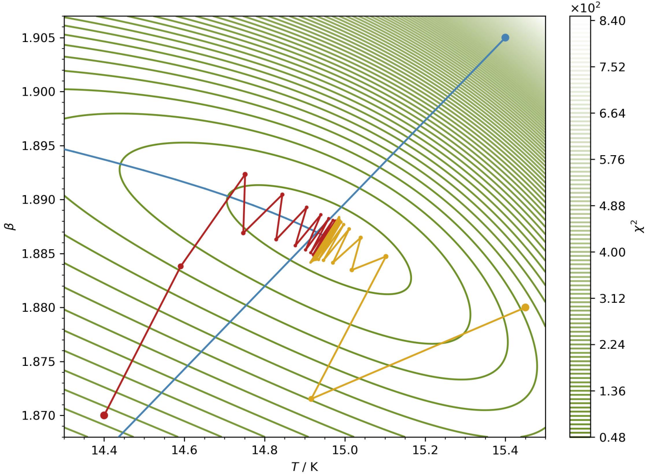
Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm
- *Fixed Rate:* $\alpha = 10^{-9}$
  - very smooth, but does not converge, step size too small



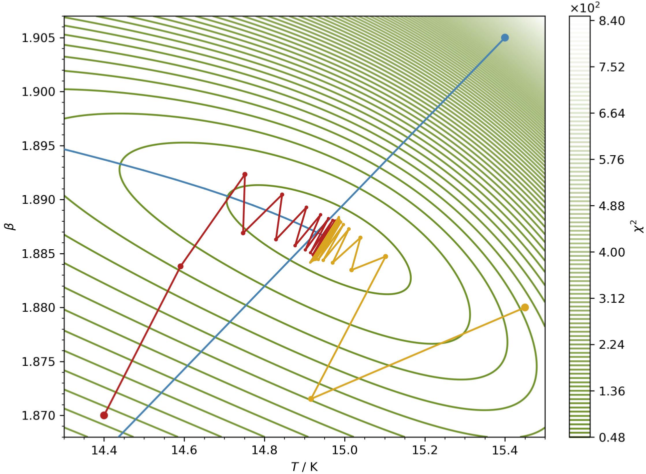
Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm
- *Fixed Rate:* $\alpha = 3 \cdot 10^{-6}$



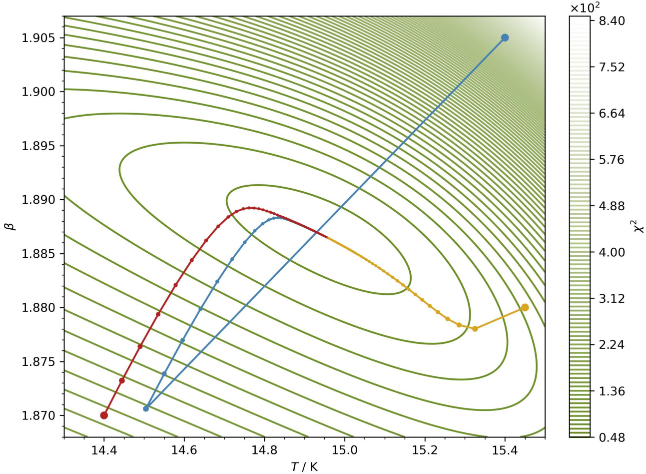
Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm
- *Fixed Rate:* $\alpha = 3 \cdot 10^{-6}$
  - oscillations, does not converge, step size too large



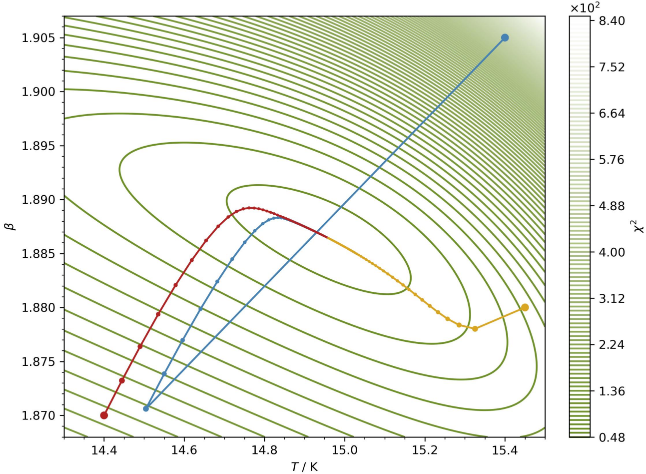
Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm
- *Fixed Rate:* $\alpha = 7 \cdot 10^{-7}$



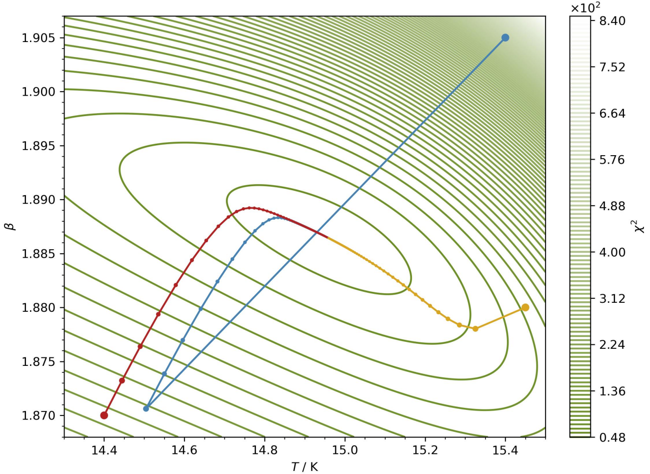
Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm
- *Fixed Rate:* $\alpha = 7 \cdot 10^{-7}$
  - this looks good, optimisers converge



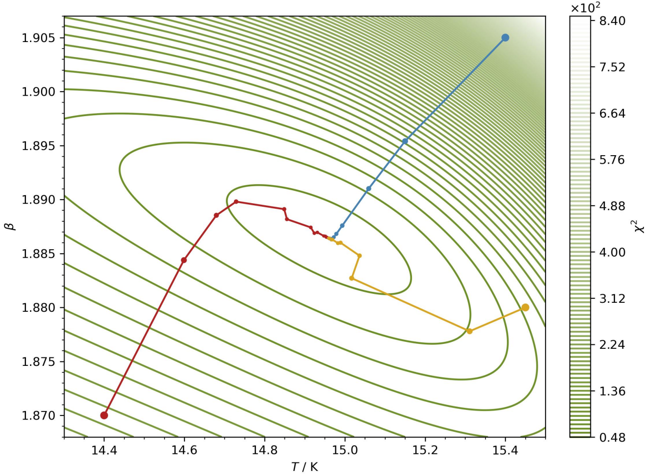
Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm
- *Fixed Rate:* $\alpha = 7 \cdot 10^{-7}$
  - $T_{400} = 14.96$ K & $\beta_{400} = 1.866$



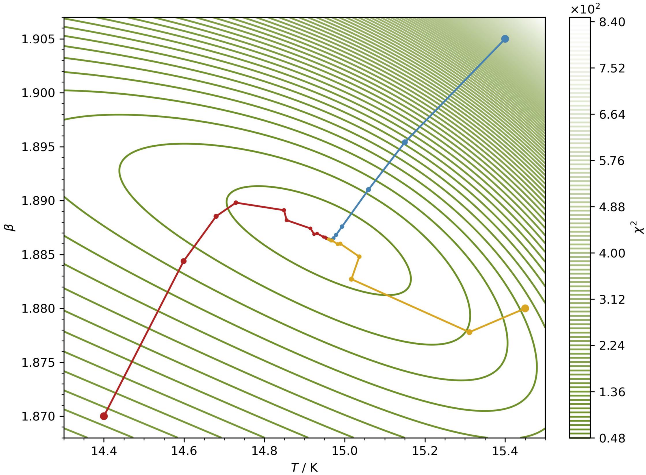
Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm
- *Fixed Rate:* $\alpha = 7 \cdot 10^{-7}$
  - $T_{400} = 14.96$ K & $\beta_{400} = 1.866$
- *Line Search:* $\alpha_0 = 10^{-4}$ & $\tau = \kappa = 0.5$

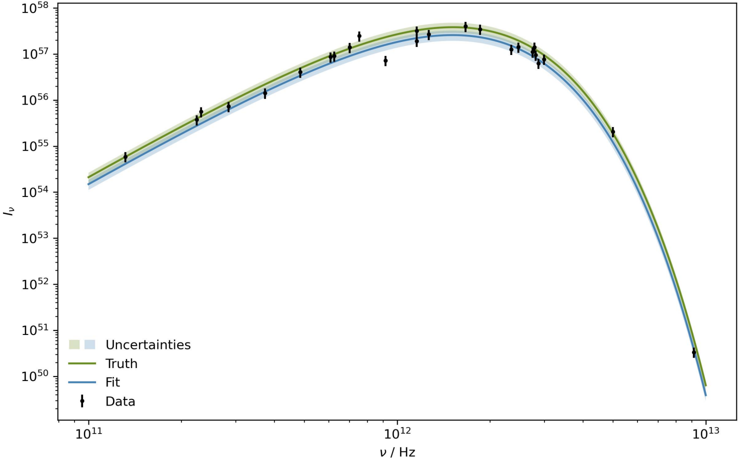


Application

- *Example:* minimise $\chi^2(\theta)$ using the gradient descent algorithm
- *Fixed Rate:* $\alpha = 7 \cdot 10^{-7}$
  - $T_{400} = 14.96$ K & $\beta_{400} = 1.866$
- *Line Search:* $\alpha_0 = 10^{-4}$ & $\tau = \kappa = 0.5$
  - $T_{40} = 14.96$ K & $\beta_{40} = 1.866$



Result

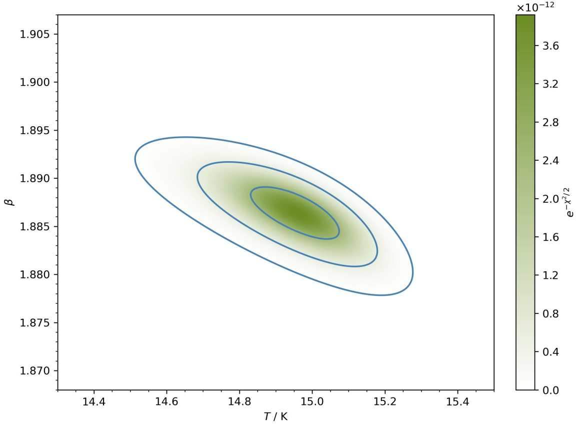

10^{58}
10^{57}
10^{56}
10^{55}
10^{54}
10^{53}
10^{52}
10^{51}
10^{50}
I_ν
Uncertainties
Truth
Fit
Data
10^{11}
10^{12}
10^{13}
ν / Hz

Overview

Estimating Covariance

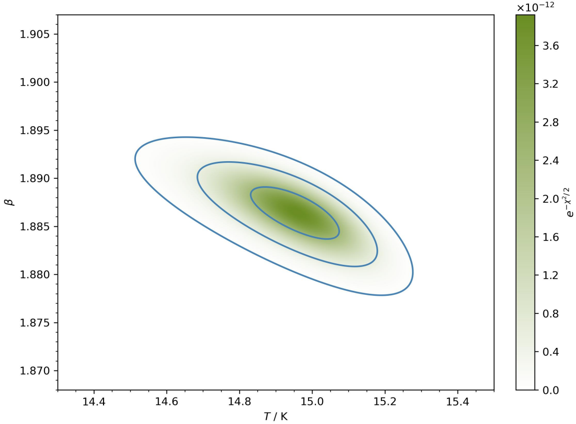
Estimating Covariance

- *Gridded Posterior Contours*



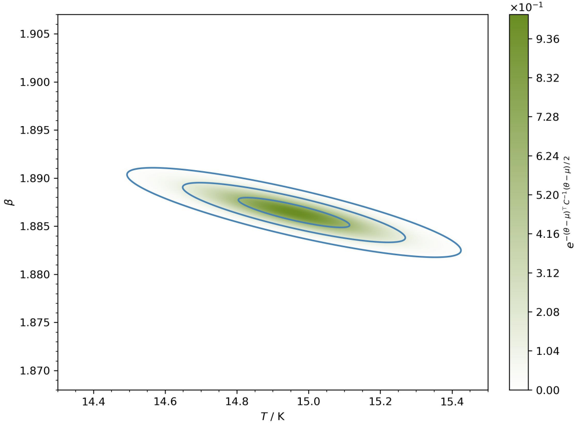
Estimating Covariance

- *Gridded Posterior Contours*
- *Gaussian Approximations*
  $x \sim N(\mu, \Sigma) \propto \exp\left(-\frac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu)\right)$



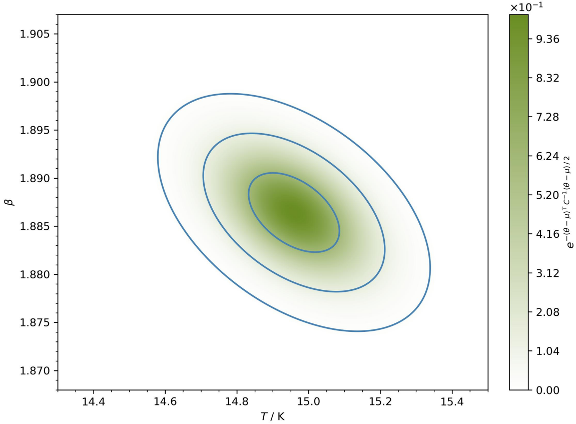
Estimating Covariance

- *Gridded Posterior Contours*
- *Gaussian Approximations*
  $x \sim N(\mu, \Sigma) \propto \exp\left(-\frac{1}{2}(x-\mu)^{\mathrm{T}}\Sigma^{-1}(x-\mu)\right)$
  - *Inverse Fisher Information*
    $I(\theta) = \mathrm{Var}[\nabla\ell] = E\left[(\nabla\ell)(\nabla\ell)^{\mathrm{T}}\right] = -E[H\ell]$
    with $\ell = \log P$



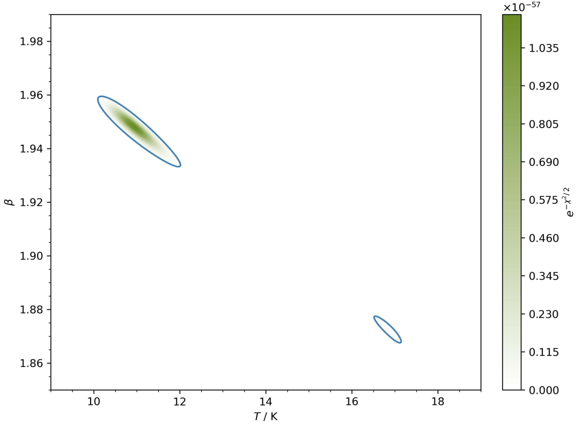
Estimating Covariance

- *Gridded Posterior Contours*
- *Gaussian Approximations*
  $x \sim N(\mu, \Sigma) \propto \exp\left(-\frac{1}{2}(x-\mu)^{\mathsf{T}}\Sigma^{-1}(x-\mu)\right)$
  - *Inverse Fisher Information*
    $I(\theta) = \mathrm{Var}[\nabla\ell] = E\left[(\nabla\ell)(\nabla\ell)^{\mathsf{T}}\right] = -E[H\ell]$
    with $\ell = \log P$
  - Integrating Posterior P for
    $\mathrm{Cov}(\theta_1, \theta_2) = E\left[(\theta_1 - E[\theta_1])(\theta_2 - E[\theta_2])\right]$



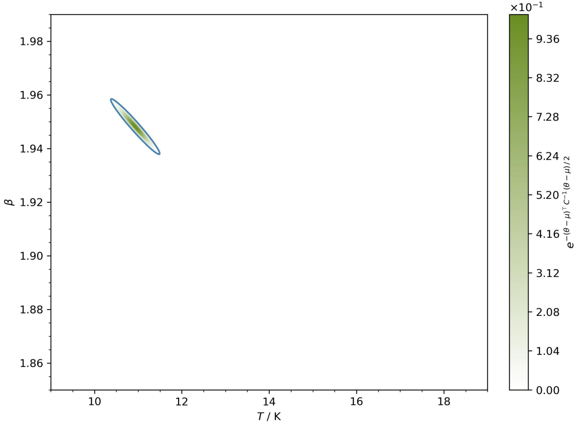
Estimating Covariance

- *Gridded Posterior Contours*
- *Gaussian Approximations*
  $x \sim N(\mu, \Sigma) \propto \exp\left(-\frac{1}{2}(x-\mu)^\mathsf{T}\Sigma^{-1}(x-\mu)\right)$
  - *Inverse Fisher Information*
    $I(\theta) = \mathrm{Var}[\nabla\ell] = E\left[(\nabla\ell)(\nabla\ell)^\mathsf{T}\right] = -E[H\ell]$
    with $\ell = \log P$
  - Integrating Posterior P for
    $\mathrm{Cov}(\theta_1, \theta_2) = E\left[(\theta_1 - E[\theta_1])(\theta_2 - E[\theta_2])\right]$



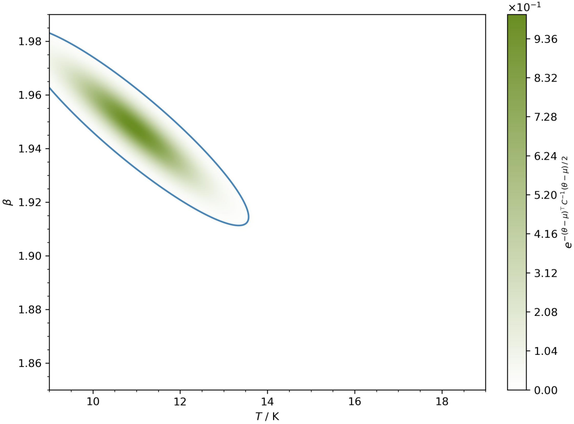
Estimating Covariance

- *Gridded Posterior Contours*
- *Gaussian Approximations*
  $x \sim N(\mu, \Sigma) \propto \exp\left(-\frac{1}{2}(x-\mu)^{\mathsf{T}}\Sigma^{-1}(x-\mu)\right)$
  - *Inverse Fisher Information*
    $I(\theta) = \mathrm{Var}[\nabla\ell] = E\left[(\nabla\ell)(\nabla\ell)^{\mathsf{T}}\right] = -E[H\ell]$
    with $\ell = \log P$
  - Integrating Posterior P for
    $\mathrm{Cov}(\theta_1, \theta_2) = E\left[(\theta_1 - E[\theta_1])(\theta_2 - E[\theta_2])\right]$



Estimating Covariance

- *Gridded Posterior Contours*
- *Gaussian Approximations*
  $x \sim N(\mu, \Sigma) \propto \exp\left(-\frac{1}{2}(x-\mu)^{\mathsf{T}}\Sigma^{-1}(x-\mu)\right)$
  - *Inverse Fisher Information*
    $I(\theta) = \mathrm{Var}[\nabla\ell] = E\left[(\nabla\ell)(\nabla\ell)^{\mathsf{T}}\right] = -E[H\ell]$
    with $\ell = \log P$
  - Integrating Posterior P for
    $\mathrm{Cov}(\theta_1, \theta_2) = E\left[(\theta_1 - E[\theta_1])(\theta_2 - E[\theta_2])\right]$



Limitations & Curses

Limitations & Curses

- *Convergence & Silent Failures:*

Limitations & Curses

- *Convergence & Silent Failures:*
  - algorithm continues until convergence threshold is met or optimisation fails

Limitations & Curses

- *Convergence & Silent Failures:*
  - algorithm continues until convergence threshold is met or optimisation fails
  - check output for which termination reason applies

Limitations & Curses

- *Convergence & Silent Failures:*
  - algorithm continues until convergence threshold is met or optimisation fails
  - check output for which termination reason applies
  - verify that parameters have actually changed from initial guess

Limitations & Curses

- *Convergence & Silent Failures:*
  - algorithm continues until convergence threshold is met or optimisation fails
  - check output for which termination reason applies
  - verify that parameters have actually changed from initial guess
- *Dimensionality:*

Limitations & Curses

- *Convergence & Silent Failures:*
  - algorithm continues until convergence threshold is met or optimisation fails
  - check output for which termination reason applies
  - verify that parameters have actually changed from initial guess
- *Dimensionality:*
  - volume and distance become somewhat meaningless in higher dimensions

Limitations & Curses

- *Convergence & Silent Failures:*
  - algorithm continues until convergence threshold is met or optimisation fails
  - check output for which termination reason applies
  - verify that parameters have actually changed from initial guess
- *Dimensionality:*
  - volume and distance become somewhat meaningless in higher dimensions
  - impossible to determine where to step or which parameter is important

Limitations & Curses

- *Convergence & Silent Failures:*
  - algorithm continues until convergence threshold is met or optimisation fails
  - check output for which termination reason applies
  - verify that parameters have actually changed from initial guess
- *Dimensionality:*
  - volume and distance become somewhat meaningless in higher dimensions
  - impossible to determine where to step or which parameter is important
  - gridding uniformly in linear or logarithmic space is expensive but a useful option

Limitations & Curses

- *Convergence & Silent Failures:*
  - algorithm continues until convergence threshold is met or optimisation fails
  - check output for which termination reason applies
  - verify that parameters have actually changed from initial guess
- *Dimensionality:*
  - volume and distance become somewhat meaningless in higher dimensions
  - impossible to determine where to step or which parameter is important
  - gridding uniformly in linear or logarithmic space is expensive but a useful option
  - fixing some parameters or optimising independently are potential solutions

Limitations & Curses

Limitations & Curses

- *Scaling:*

Limitations & Curses

- *Scaling:*
  - parameter scalings of different magnitudes produce pathological stopping criteria

Limitations & Curses

- *Scaling:*
  - parameter scalings of different magnitudes produce pathological stopping criteria
- *Nonconvexity:*

Limitations & Curses

- *Scaling:*
  - parameter scalings of different magnitudes produce pathological stopping criteria
- *Nonconvexity:*
  - convergence does not guarantee that the global optimum has been found

Limitations & Curses

- *Scaling:*
  - parameter scalings of different magnitudes produce pathological stopping criteria
- *Nonconvexity:*
  - convergence does not guarantee that the global optimum has been found
  - basin hopping or simulated annealing as potential fixes

Limitations & Curses

- *Scaling:*
  - parameter scalings of different magnitudes produce pathological stopping criteria
- *Nonconvexity:*
  - convergence does not guarantee that the global optimum has been found
  - basin hopping or simulated annealing as potential fixes
- *Initialisation:*

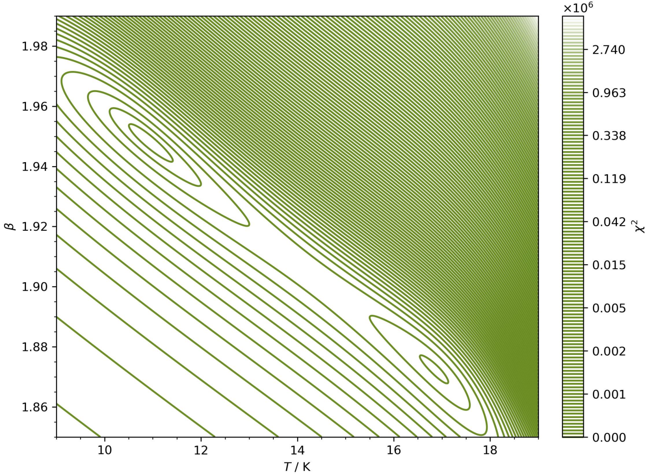
Limitations & Curses

- *Scaling:*
  - parameter scalings of different magnitudes produce pathological stopping criteria
- *Nonconvexity:*
  - convergence does not guarantee that the global optimum has been found
  - basin hopping or simulated annealing as potential fixes
- *Initialisation:*
  - optimisation can converge to different local optima depending on starting point and algorithm

Limitations & Curses

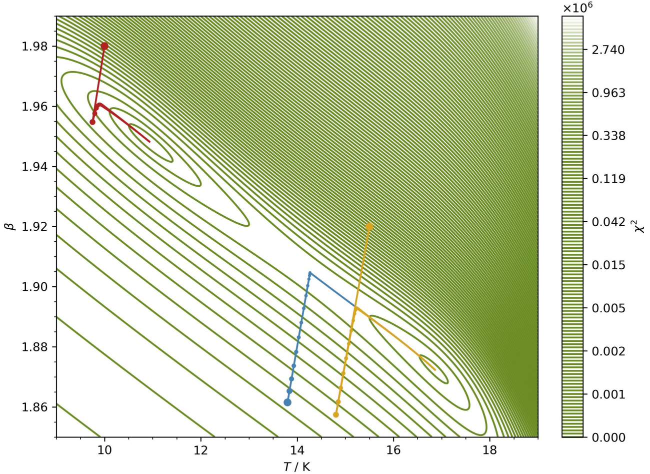
Limitations & Curses

- *Example:* T_0 = 13.8 K & β_0 = 1.8615



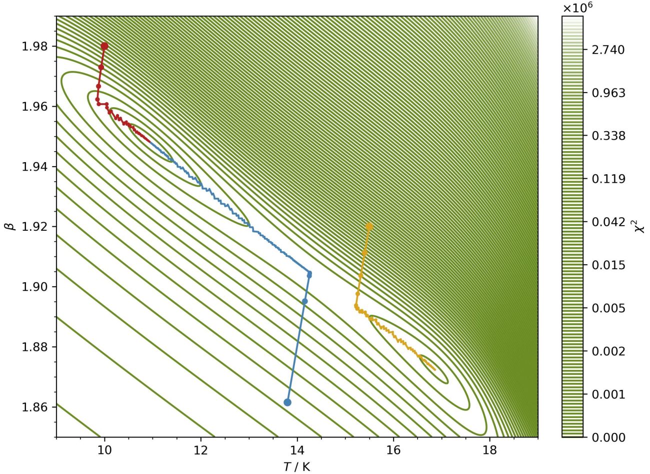
Limitations & Curses

- *Example:* T_0 = 13.8 K & β_0 = 1.8615
    - Fixed Rate Gradient Descent:
      T_{2700} = 16.86 K & β_{2700} = 1.872



Limitations & Curses

- *Example:* T_0 = 13.8 K & β_0 = 1.8615
  - Fixed Rate Gradient Descent:
    T_{2700} = 16.86 K & β_{2700} = 1.872
  - Line Search Gradient Descent:
    T_{660} = 10.93 K & β_{660} = 1.948



Overview

- **Probability Refresher**
  - Definitions & Notation
  - Random Numbers
- **Optimisation**
  - Common Methods
  - Problems
- **Markov Chain Monte Carlo**
  - Popular Algorithms
  - Tuning & Diagnosing
  - Alternatives

Motivation

Motivation

- *Optimisation:* finds a point estimate of optimal model parameters

Motivation

- *Optimisation:* finds a point estimate of optimal model parameters
- *Monte Carlo Sampling:* dynamically explores the parameter space distribution

Motivation

- *Optimisation:* finds a point estimate of optimal model parameters
- *Monte Carlo Sampling:* dynamically explores the parameter space distribution
    - quote asymmetric percentiles

Motivation

- *Optimisation:* finds a point estimate of optimal model parameters
- *Monte Carlo Sampling:* dynamically explores the parameter space distribution
  - quote asymmetric percentiles
  - estimate expectation values by integrating

Motivation

- *Optimisation:* finds a point estimate of optimal model parameters
- *Monte Carlo Sampling:* dynamically explores the parameter space distribution
  - quote asymmetric percentiles
  - estimate expectation values by integrating
  - uncertainty propagation to the model function

Motivation

- *Optimisation:* finds a point estimate of optimal model parameters
- *Monte Carlo Sampling:* dynamically explores the parameter space distribution
    - quote asymmetric percentiles
    - estimate expectation values by integrating
    - uncertainty propagation to the model function
    - simulate complex measurement processes and quantify the total convolution

Markov Chain

Markov Chain

- *Properties:*

Markov Chain

- *Properties:*
  - sequence generated by stochastic transitions

Markov Chain

- *Properties:*
  - sequence generated by stochastic transitions
  - transition probabilities depend only on the initial and final state

Markov Chain

- *Properties:*
  - sequence generated by stochastic transitions
  - transition probabilities depend only on the initial and final state
- *Detailed Balance:*

Markov Chain

- *Properties:*
  - sequence generated by stochastic transitions
  - transition probabilities depend only on the initial and final state
- *Detailed Balance:*
  - any chain belonging to a Markov Process represents a reversible sequence of events

Markov Chain

- *Properties:*
  - sequence generated by stochastic transitions
  - transition probabilities depend only on the initial and final state
- *Detailed Balance:*
  - any chain belonging to a Markov Process represents a reversible sequence of events
  - when walking along a stationary distribution $f(x)$ the following must hold for every transition probability $M_{i\to j}$ from state x_i to x_j

$$M_{i\to j} f(x_i) = M_{j\to i} f(x_j)$$

Markov Chain

- *Properties:*
  - sequence generated by stochastic transitions
  - transition probabilities depend only on the initial and final state
- *Detailed Balance:*
  - any chain belonging to a Markov Process represents a reversible sequence of events
  - when walking along a stationary distribution $f(x)$ the following must hold for every transition probability $M_{i \to j}$ from state x_i to x_j

$$M_{i \to j} f(x_i) = M_{j \to i} f(x_j)$$

  - then the average behaviour of this single process reproduces that over the entire ensemble

Monte Carlo

Monte Carlo

- *Randomness:*

Monte Carlo

- *Randomness:*
  - each proposed next step is accepted or discarded based on a probabilistic condition

Monte Carlo

- *Randomness:*
  - each proposed next step is accepted or discarded based on a probabilistic condition
  - if the uncerlying distribution $f(x_{k+1})$ is larger at the new position x_{k+1} it is updated

Monte Carlo

- *Randomness:*
  - each proposed next step is accepted or discarded based on a probabilistic condition
  - if the uncerlying distribution $f(x_{k+1})$ is larger at the new position x_{k+1} it is updated
  - else it is based on a random choice with acceptance being less likely for smaller $f(x_{k+1})$

Monte Carlo

- *Randomness:*
  - each proposed next step is accepted or discarded based on a probabilistic condition
  - if the uncerlying distribution $f(x_{k+1})$ is larger at the new position x_{k+1} it is updated
  - else it is based on a random choice with acceptance being less likely for smaller $f(x_{k+1})$
- *Rules:*

Monte Carlo

- *Randomness:*
  - each proposed next step is accepted or discarded based on a probabilistic condition
  - if the uncerlying distribution $f(x_{k+1})$ is larger at the new position x_{k+1} it is updated
  - else it is based on a random choice with acceptance being less likely for smaller $f(x_{k+1})$
- *Rules:*
  - the proposal distribution is allowed to violate detailed balance only if it is corrected in the decision between acceptance and rejection

Monte Carlo

- *Randomness:*
  - each proposed next step is accepted or discarded based on a probabilistic condition
  - if the uncerlying distribution $f(x_{k+1})$ is larger at the new position x_{k+1} it is updated
  - else it is based on a random choice with acceptance being less likely for smaller $f(x_{k+1})$
- *Rules:*
  - the proposal distribution is allowed to violate detailed balance only if it is corrected in the decision between acceptance and rejection
  - unlike the underlying posterior that we sample, the proposal must be properly normalized

Overview

Markov Chain Monte Carlo

Markov Chain Monte Carlo

- *Metropolis:*

Markov Chain Monte Carlo

- *Metropolis:*
  - a proposal x_j is drawn from a fixed step size distribution such as a normal or uniform one that is symmetric in each coordinate $g(x_j|x_i) = g(x_i|x_j)$

Markov Chain Monte Carlo

- *Metropolis:*
  - a proposal x_j is drawn from a fixed step size distribution such as a normal or uniform one that is symmetric in each coordinate $g(x_j|x_i) = g(x_i|x_j)$
  - this satisifies detailed balance $M_{i\to j} f(x_i) = M_{j\to i} f(x_j)$ with

$$M_{i\to j} = \min\left(1, \frac{f(x_j)}{f(x_i)}\right)$$

Markov Chain Monte Carlo

- *Metropolis:*
  - a proposal x_j is drawn from a fixed step size distribution such as a normal or uniform one that is symmetric in each coordinate $g(x_j|x_i) = g(x_i|x_j)$
  - this satisifies detailed balance $M_{i\to j} f(x_i) = M_{j\to I} f(x_j)$ with

$$M_{i\to j} = \min\left(1\,,\,\frac{f(x_j)}{f(x_i)}\right)$$

  - generate a random number $\xi \sim U(0,1)$ and decide according to

$$x_{i+1} = \begin{cases} x_j & \xi \leq M_{i\to j} \\ x_i & \xi > M_{i\to j} \end{cases}$$

Markov Chain Monte Carlo

Markov Chain Monte Carlo

- *Metropolis-Hastings:*

Markov Chain Monte Carlo

- *Metropolis-Hastings:*
  - a proposal x_j is drawn from an asymmetric distribution $g(x_j|x_i) \neq g(x_i|x_j)$

Markov Chain Monte Carlo

- *Metropolis-Hastings:*
  - a proposal x_j is drawn from an asymmetric distribution $g(x_j|x_i) \neq g(x_i|x_j)$
  - this itself violates detailed balance but is fixed by $M_{i\to j} f(x_i) g(x_j|x_i) = M_{j\to i} f(x_j) g(x_i|x_j)$ with

$$M_{i\to j} = \min\left(1, \frac{f(x_j) g(x_i|x_j)}{f(x_i) g(x_j|x_i)}\right)$$

Markov Chain Monte Carlo

- *Metropolis-Hastings:*
  - a proposal x_j is drawn from an asymmetric distribution $g(x_j|x_i) \neq g(x_i|x_j)$
  - this itself violates detailed balance but is fixed by $M_{i\to j} f(x_i) g(x_j|x_i) = M_{j\to i} f(x_j) g(x_i|x_j)$ with

$$M_{i\to j} = \min\left(1\,, \frac{f(x_j)g(x_i|x_j)}{f(x_i)g(x_j|x_i)}\right)$$

  - generate a random number $\xi \sim U(0,1)$ and decide according to

$$x_{i+1} = \begin{cases} x_j & \xi \leq M_{i\to j} \\ x_i & \xi > M_{i\to j} \end{cases}$$

Markov Chain Monte Carlo

Markov Chain Monte Carlo

- *Comparison:*

Markov Chain Monte Carlo

- *Comparison:*
  - the Metropolis-Hastings is more flexible than the Metropolis algorithm since it can use asymmetric proposal distributions

Markov Chain Monte Carlo

- *Comparison:*
  - the Metropolis-Hastings is more flexible than the Metropolis algorithm since it can use asymmetric proposal distributions
  - asymmetric proposals allow efficient handling of bounded parameter spaces

Markov Chain Monte Carlo

- *Comparison:*
  - the Metropolis-Hastings is more flexible than the Metropolis algorithm since it can use asymmetric proposal distributions
  - asymmetric proposals allow efficient handling of bounded parameter spaces
  - proposal distribution shapes can be tailored to the sampled posterior to speed up exploration of the space and achieve good convergence

Markov Chain Monte Carlo

- *Comparison:*
  - the Metropolis-Hastings is more flexible than the Metropolis algorithm since it can use asymmetric proposal distributions
  - asymmetric proposals allow efficient handling of bounded parameter spaces
  - proposal distribution shapes can be tailored to the sampled posterior to speed up exploration of the space and achieve good convergence
  - realistic cases, especially when priors are included, are generally not symmetric or unbounded, therefore benefitting from the inbuilt bias correction to ensure correct sampling of the target distribution

Advanced Methods

Advanced Methods

- *Affine-Invariant:*

Advanced Methods

- *Affine-Invariant:*
  - multiple walkers that spread out into parameter space controlled by stretch parameter

Advanced Methods

- *Affine-Invariant:*
  - multiple walkers that spread out into parameter space controlled by stretch parameter
  - biased proposals are made along a line connecting randomly paired walkers and corrected in the decision step

Advanced Methods

- *Affine-Invariant:*
    - multiple walkers that spread out into parameter space controlled by stretch parameter
    - biased proposals are made along a line connecting randomly paired walkers and corrected in the decision step
    - very efficient at sampling correlated parameters due to reduced hyperparameters

Advanced Methods

- *Affine-Invariant:*
  - multiple walkers that spread out into parameter space controlled by stretch parameter
  - biased proposals are made along a line connecting randomly paired walkers and corrected in the decision step
  - very efficient at sampling correlated parameters due to reduced hyperparameters
  - suitable in low dimensional parameter spaces where no gradient information is known

Advanced Methods

Advanced Methods

- *Adjusted Langevin:*

Advanced Methods

- *Adjusted Langevin:*
  - uses gradient of logarithmic likelihood or score function as a drift force pushing samples towards higher probability

Advanced Methods

- *Adjusted Langevin:*
  - uses gradient of logarithmic likelihood or score function as a drift force pushing samples towards higher probability
  - a small Gaussian noise term is added to act as a random force to allow exploration of the distribution

Advanced Methods

- *Adjusted Langevin:*
  - uses gradient of logarithmic likelihood or score function as a drift force pushing samples towards higher probability
  - a small Gaussian noise term is added to act as a random force to allow exploration of the distribution
  - bias is corrected with a Metropolis-Hastings decision step

Advanced Methods

- *Adjusted Langevin:*
  - uses gradient of logarithmic likelihood or score function as a drift force pushing samples towards higher probability
  - a small Gaussian noise term is added to act as a random force to allow exploration of the distribution
  - bias is corrected with a Metropolis-Hastings decision step
  - application in gradient estimation and diffusion lerning

Advanced Methods

Advanced Methods

- *Hamilton:*

Advanced Methods

- *Hamilton:*
  - treat target function as potential energy, introduce random momentum term for kinetic energy

Advanced Methods

- *Hamilton:*
  - treat target function as potential energy, introduce random momentum term for kinetic energy
  - definition in canonical distribution ensures separability so that x and p are independent

Advanced Methods

- *Hamilton:*
  - treat target function as potential energy, introduce random momentum term for kinetic energy
  - definition in canonical distribution ensures separability so that x and p are independent
  - for each step, accurately integrate momentum some set amount of steps by using the gradient, conditioned by the Hamiltonian or total energy remaining constant

Advanced Methods

- *Hamilton:*
  - treat target function as potential energy, introduce random momentum term for kinetic energy
  - definition in canonical distribution ensures separability so that x and p are independent
  - for each step, accurately integrate momentum some set amount of steps by using the gradient, conditioned by the Hamiltonian or total energy remaining constant
  - check proposal with Metropolis decision metric to account for numerical integration errors

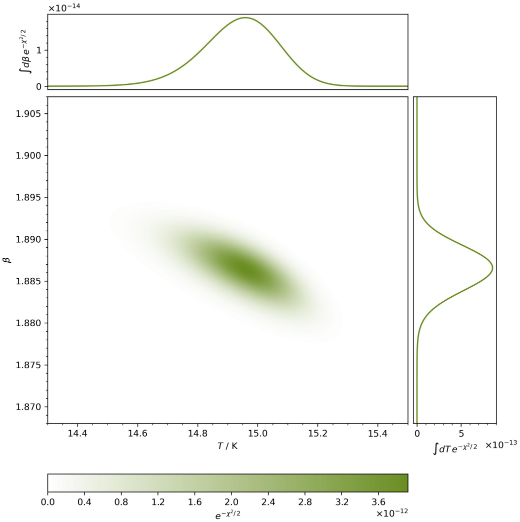
Advanced Methods

- *Hamilton:*
  - treat target function as potential energy, introduce random momentum term for kinetic energy
  - definition in canonical distribution ensures separability so that x and p are independent
  - for each step, accurately integrate momentum some set amount of steps by using the gradient, conditioned by the Hamiltonian or total energy remaining constant
  - check proposal with Metropolis decision metric to account for numerical integration errors
  - permits very long directed jumps that drastically reduce the correlation of the chain

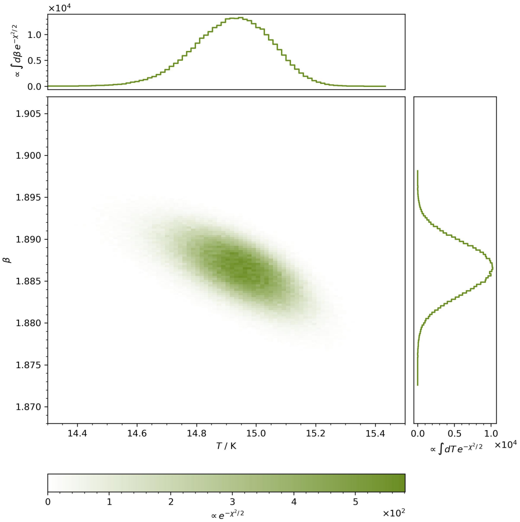
Advanced Methods

- *Hamilton:*
    - treat target function as potential energy, introduce random momentum term for kinetic energy
    - definition in canonical distribution ensures separability so that x and p are independent
    - for each step, accurately integrate momentum some set amount of steps by using the gradient, conditioned by the Hamiltonian or total energy remaining constant
    - check proposal with Metropolis decision metric to account for numerical integration errors
    - permits very long directed jumps that drastically reduce the correlation of the chain
    - efficient for many dimensions with very high acceptance ratios

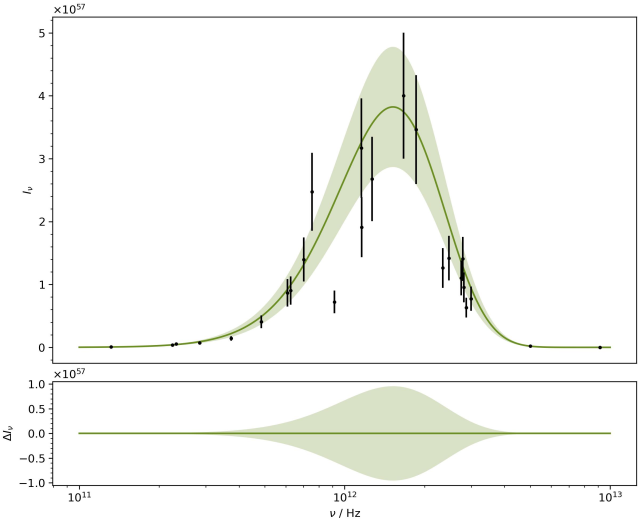
Posterior


×10^−14
∫dβ e^−χ²/2
1
0
1.905
1.900
1.895
1.890
1.885
1.880
1.875
1.870
β
14.4
14.6
14.8
15.0
15.2
15.4
T / K
0
5
∫dT e^−χ²/2
×10^−13
0.0
0.4
0.8
1.2
1.6
2.0
2.4
2.8
3.2
3.6
e^−χ²/2
×10^−12

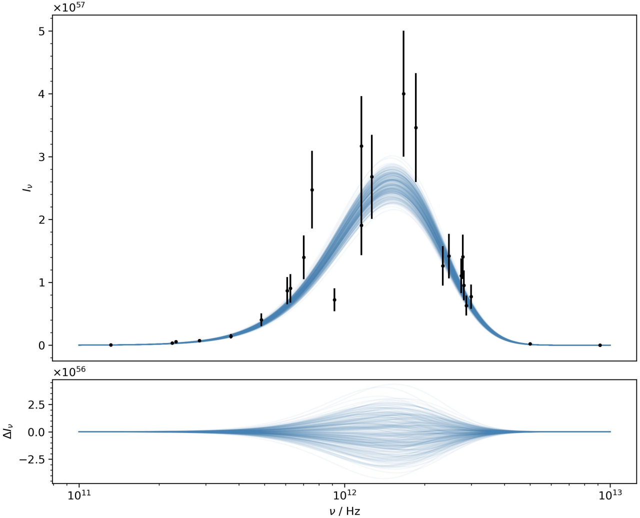
Posterior


×10⁴
∝ ∫dβ e^{−χ²/2}
1.0
0.5
0.0
β
1.905
1.900
1.895
1.890
1.885
1.880
1.875
1.870
14.4
14.6
14.8
15.0
15.2
15.4
T / K
0.0
0.5
1.0
∝ ∫dT e^{−χ²/2}
×10⁴
0
1
2
3
4
5
∝ e^{−χ²/2}
×10²

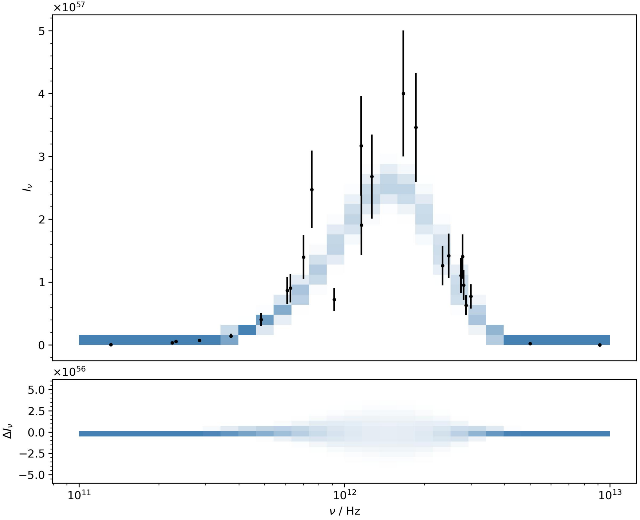
Comparison


×10^57
5
4
3
2
1
0
I_ν
×10^57
1.0
0.5
0.0
−0.5
−1.0
ΔI_ν
10^{11}
10^{12}
10^{13}
ν / Hz

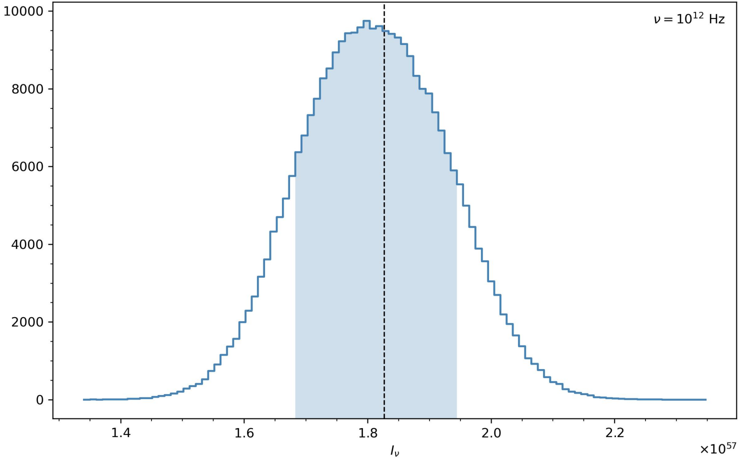
Comparison



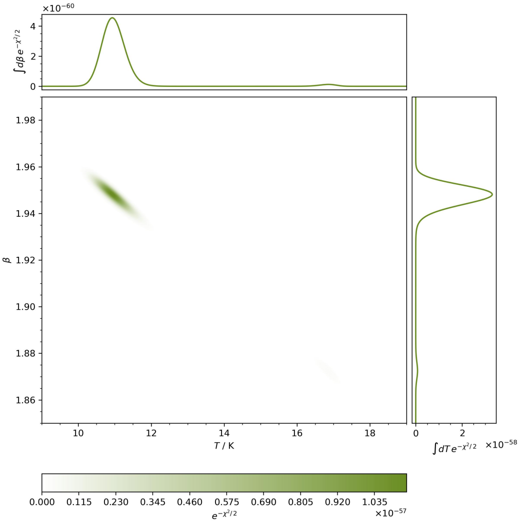
Comparison


×10^57
5
4
3
2
1
0
I_ν
×10^56
5.0
2.5
0.0
−2.5
−5.0
ΔI_ν
10^{11}
10^{12}
10^{13}
ν / Hz

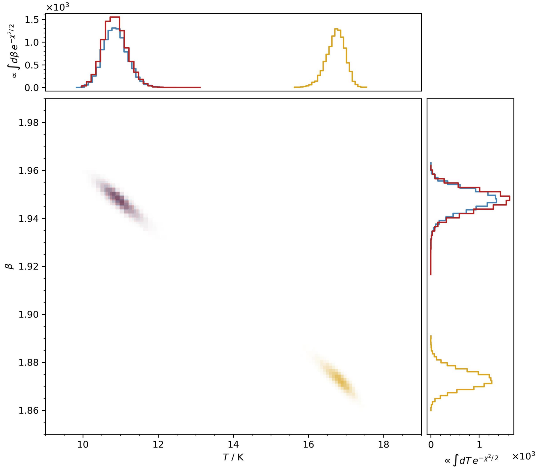
Error Propagation



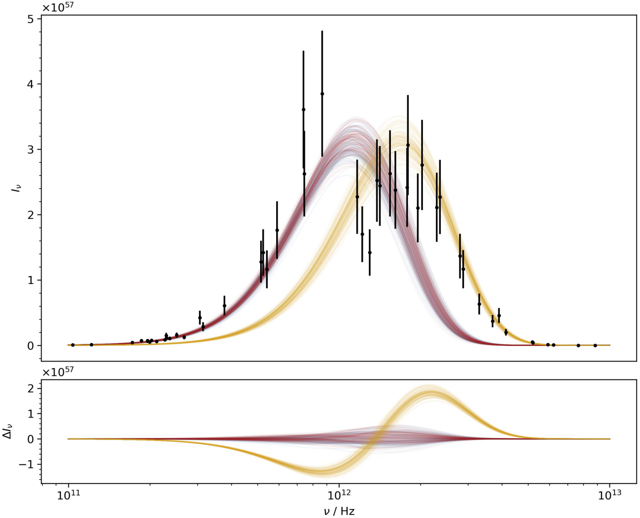
Posterior


×10⁻⁶⁰
$\int d\beta\, e^{-\chi^2/2}$
4
2
0
1.98
1.96
1.94
1.92
1.90
1.88
1.86
β
10
12
14
16
18
T / K
0
2
$\int dT\, e^{-\chi^2/2}$ ×10⁻⁵⁸
0.000
0.115
0.230
0.345
0.460
0.575
0.690
0.805
0.920
1.035
$e^{-\chi^2/2}$
×10⁻⁵⁷

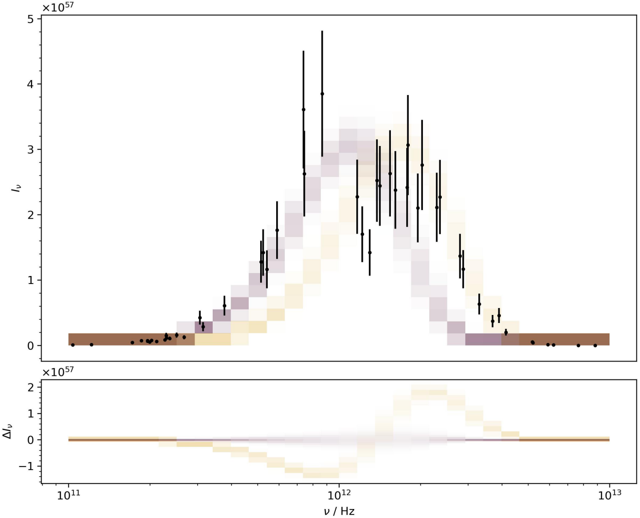
Posterior


×10^3
1.5
1.0
0.5
0.0
∝ ∫dβ e^{−χ²/2}
β
1.98
1.96
1.94
1.92
1.90
1.88
1.86
10
12
14
16
18
T / K
0
1
∝ ∫dT e^{−χ²/2}
×10^3

Comparison



Comparison



Overview

Sampling Efficiency

Sampling Efficiency

- *Low Acceptance Rate:* $\lesssim 30\%$

Sampling Efficiency

- *Low Acceptance Rate:* $\lesssim 30\%$
  - only small fraction of significant part of target explored

Sampling Efficiency

- *Low Acceptance Rate:* $\lesssim$ 30%
  - only small fraction of significant part of target explored
  - few points actually represent the distribution

Sampling Efficiency

- *Low Acceptance Rate:* $\lesssim 30\%$
  - only small fraction of significant part of target explored
  - few points actually represent the distribution
  - step size too large

Sampling Efficiency

- *Low Acceptance Rate:* $\lesssim 30\%$
  - only small fraction of significant part of target explored
  - few points actually represent the distribution
  - step size too large
- *High Acceptance Rate:* $\gtrsim 70\%$

Sampling Efficiency

- *Low Acceptance Rate:* $\lesssim 30\%$
  - only small fraction of significant part of target explored
  - few points actually represent the distribution
  - step size too large
- *High Acceptance Rate:* $\gtrsim 70\%$
  - many points close to each other

Sampling Efficiency

- *Low Acceptance Rate:* $\lesssim 30\%$
  - only small fraction of significant part of target explored
  - few points actually represent the distribution
  - step size too large
- *High Acceptance Rate:* $\gtrsim 70\%$
  - many points close to each other
  - flat part of target distribution

Sampling Efficiency

- *Low Acceptance Rate:* $\lesssim 30\%$
  - only small fraction of significant part of target explored
  - few points actually represent the distribution
  - step size too large
- *High Acceptance Rate:* $\gtrsim 70\%$
  - many points close to each other
  - flat part of target distribution
  - missing large portions of relevant parameter space

Sampling Efficiency

- *Low Acceptance Rate:* $\lesssim 30\%$
  - only small fraction of significant part of target explored
  - few points actually represent the distribution
  - step size too large
- *High Acceptance Rate:* $\gtrsim 70\%$
  - many points close to each other
  - flat part of target distribution
  - missing large portions of relevant parameter space
  - step size too small

Burn In

Burn In

- *Problem:*

Burn In

- *Problem:*
  - initialisation far from significant part of target

Burn In

- *Problem:*
  - initialisation far from significant part of target
  - flat distribution, majority of points not relevant

Burn In

- *Problem:*
  - initialisation far from significant part of target
  - flat distribution, majority of points not relevant
  - no convergence to important porrtions of parameter space

Burn In

- *Problem:*
  - initialisation far from significant part of target
  - flat distribution, majority of points not relevant
  - no convergence to important porrtions of parameter space
- *Fixes:*

Burn In

- *Problem:*
  - initialisation far from significant part of target
  - flat distribution, majority of points not relevant
  - no convergence to important porrtions of parameter space
- *Fixes:*
  - use ensemble of walkers

Burn In

- *Problem:*
  - initialisation far from significant part of target
  - flat distribution, majority of points not relevant
  - no convergence to important porrtions of parameter space
- *Fixes:*
  - use ensemble of walkers
  - identify burn in phase via trace plot and reject it

Autocorrelation

Autocorrelation

- *Definition:* $\rho_t = \dfrac{\mathrm{Cov}(X_k, X_{k+t})}{\sqrt{\mathrm{Var}(X_k)\mathrm{Var}(X_{k+t})}} = \dfrac{E[(X_k - E[X_k])(X_{k+t} - E[X_{k+t}])]}{\sqrt{E[(X_k - E[X_k])^2]E[(X_{k+t} - E[X_{k+t}])^2]}}$

Autocorrelation

- *Definition:* $\rho_t = \dfrac{\mathrm{Cov}(X_k, X_{k+t})}{\sqrt{\mathrm{Var}(X_k)\mathrm{Var}(X_{k+t})}} = \dfrac{E[(X_k - E[X_k])(X_{k+t} - E[X_{k+t}])]}{\sqrt{E[(X_k - E[X_k])^2]E[(X_{k+t} - E[X_{k+t}])^2]}}$
  - Pearson correlation coefficient $\rho_t \in [-1, 1]$ for one parameter with a lagged version of itself

Autocorrelation

- *Definition:* $\rho_t = \frac{\text{Cov}(X_k, X_{k+t})}{\sqrt{\text{Var}(X_k)\text{Var}(X_{k+t})}} = \frac{E[(X_k - E[X_k])(X_{k+t} - E[X_{k+t}])]}{\sqrt{E[(X_k - E[X_k])^2]E[(X_{k+t} - E[X_{k+t}])^2]}}$
  - Pearson correlation coefficient $\rho_t \in [-1, 1]$ for one parameter with a lagged version of itself
  - Markov Chain elements are correlated with their neighbours since they are generated from their current states

Autocorrelation

- *Definition:* $\rho_t = \frac{\mathrm{Cov}(X_k, X_{k+t})}{\sqrt{\mathrm{Var}(X_k)\mathrm{Var}(X_{k+t})}} = \frac{E[(X_k - E[X_k])(X_{k+t} - E[X_{k+t}])]}{\sqrt{E[(X_k - E[X_k])^2]E[(X_{k+t} - E[X_{k+t}])^2]}}$
  - Pearson correlation coefficient $\rho_t \in [-1, 1]$ for one parameter with a lagged version of itself
  - Markov Chain elements are correlated with their neighbours since they are generated from their current states
  - effective sample size is reduced by autocorrelation scale

Autocorrelation

- *Definition:* $\rho_t = \dfrac{\mathrm{Cov}(X_k, X_{k+t})}{\sqrt{\mathrm{Var}(X_k)\mathrm{Var}(X_{k+t})}} = \dfrac{E[(X_k - E[X_k])(X_{k+t} - E[X_{k+t}])]}{\sqrt{E[(X_k - E[X_k])^2]E[(X_{k+t} - E[X_{k+t}])^2]}}$
  - Pearson correlation coefficient $\rho_t \in [-1, 1]$ for one parameter with a lagged version of itself
  - Markov Chain elements are correlated with their neighbours since they are generated from their current states
  - effective sample size is reduced by autocorrelation scale
  - compute expectation values for N samples by summing only to $M \ll N$ to avoid signal being dominated by statistical noise and after discarding burn in

Autocorrelation

Autocorrelation

- *Confidence:* $\sigma_t^2 = \frac{1}{M}\left(1 + 2\sum_{s=1}^{t} \rho_s^2\right)$

Autocorrelation

- *Confidence:* $\sigma_t^2 = \frac{1}{M}\left(1 + 2\sum_{s=1}^{t} \rho_s^2\right)$
  - Bartlett estimate for uncertainty $\boldsymbol{\sigma_t}$ of $\boldsymbol{\rho_t}$ by a truly random process

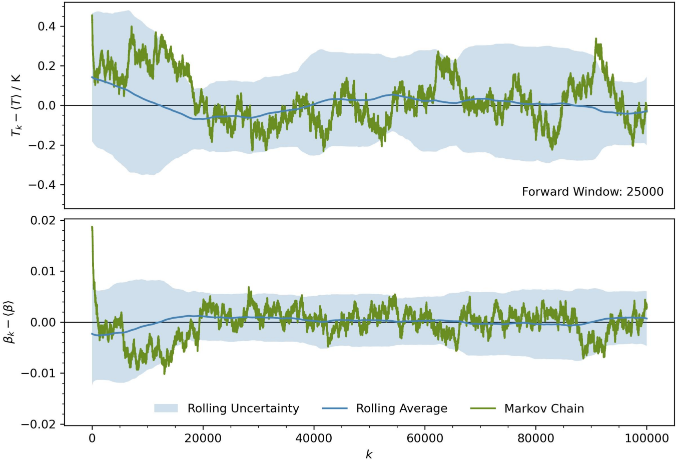
Autocorrelation

- *Confidence:* $\sigma_t^2 = \frac{1}{M}\left(1 + 2\sum_{s=1}^{t} \rho_s^2\right)$
  - Bartlett estimate for uncertainty σ_t of ρ_t by a truly random process
  - autocorrelation length l is defined by lag t after which $|\rho_t| < \sigma_t$ and therefore $\rho_t \neq 0$ no longer statistically significant

Application

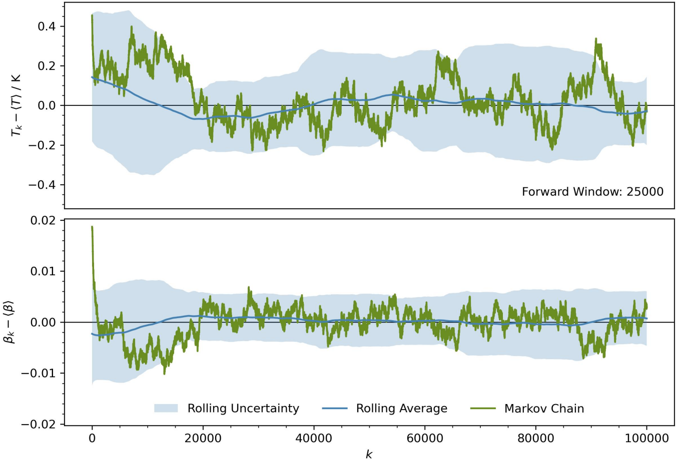
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$



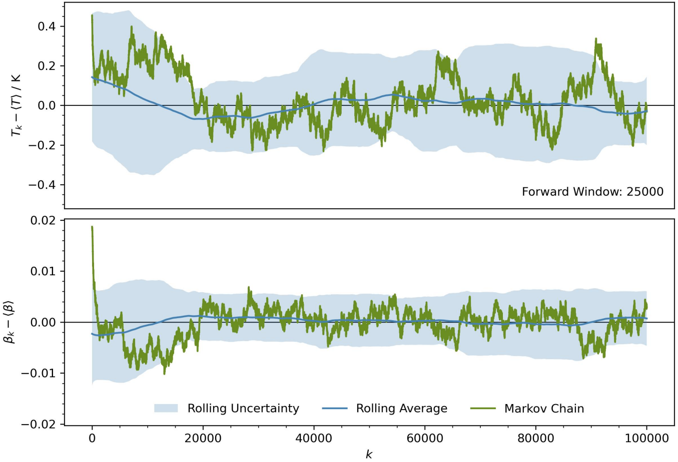
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-4}$ & acc = 97.72%



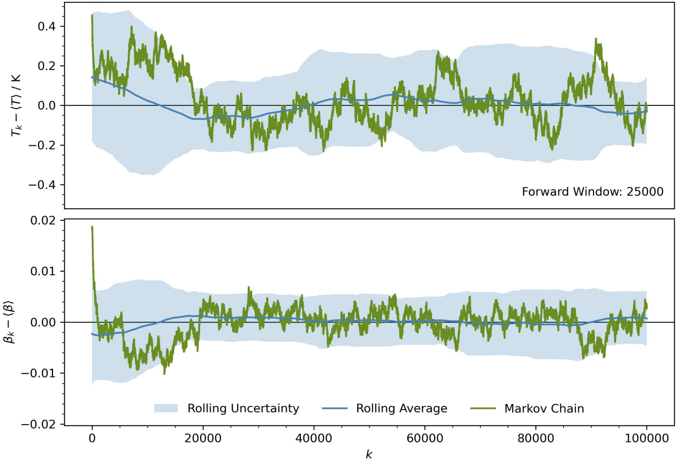
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-4}$ & acc = 97.72%
  - acceptance too high



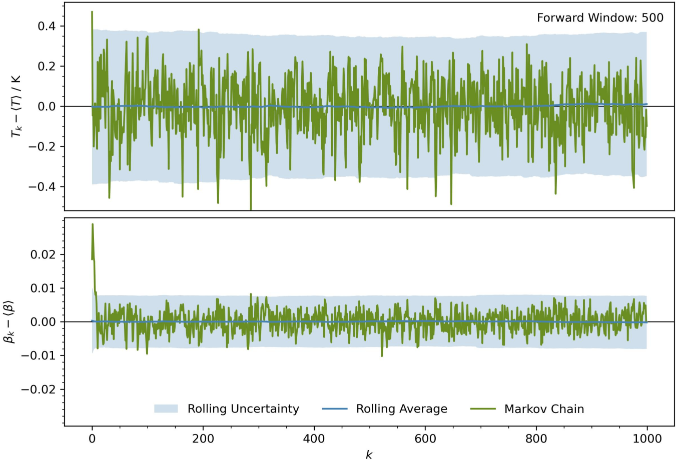
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-4}$ & acc = 97.72%
  - acceptance too high
  - step size too short



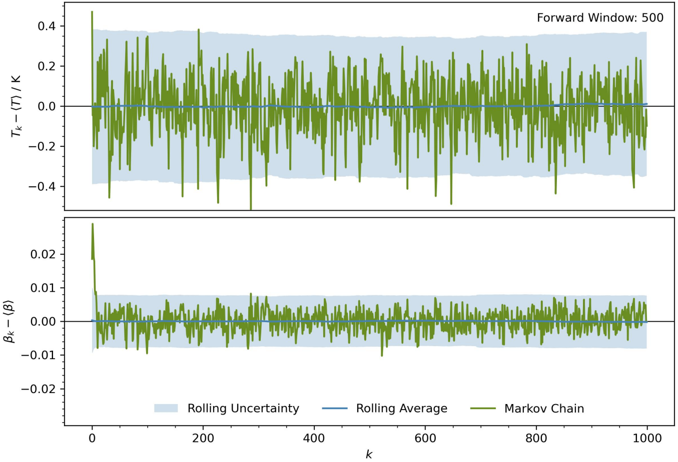
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$



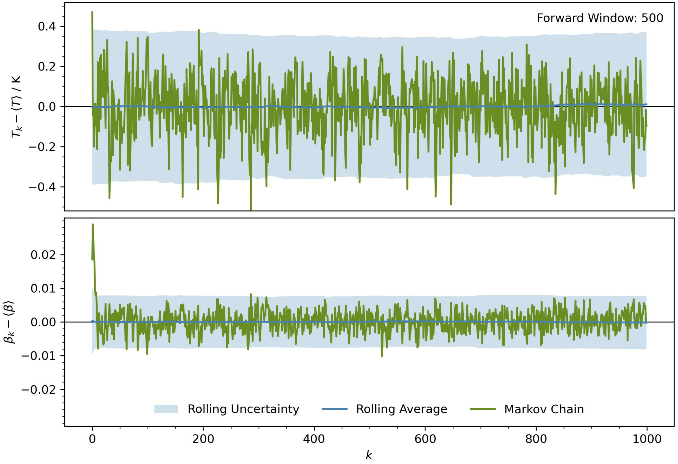
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-2}$ & acc = 12.13%



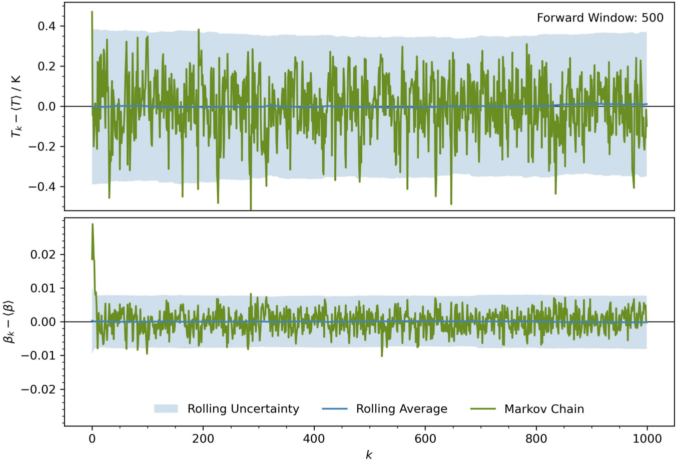
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
    - $h = 10^{-2}$ & acc = 12.13%
    - acceptance too low



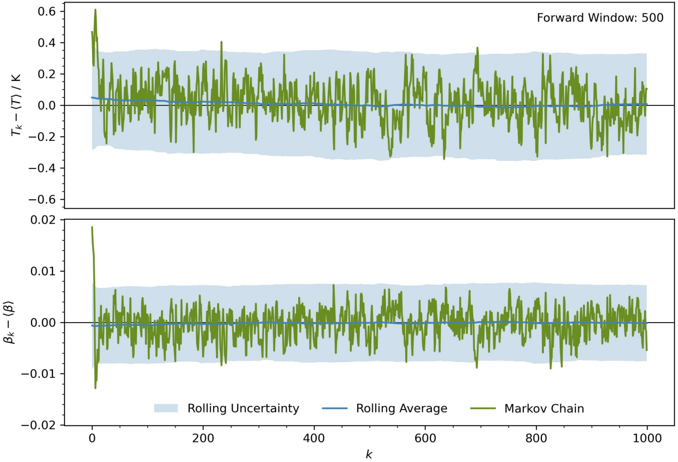
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-2}$ & acc = 12.13%
  - acceptance too low
  - step size too long



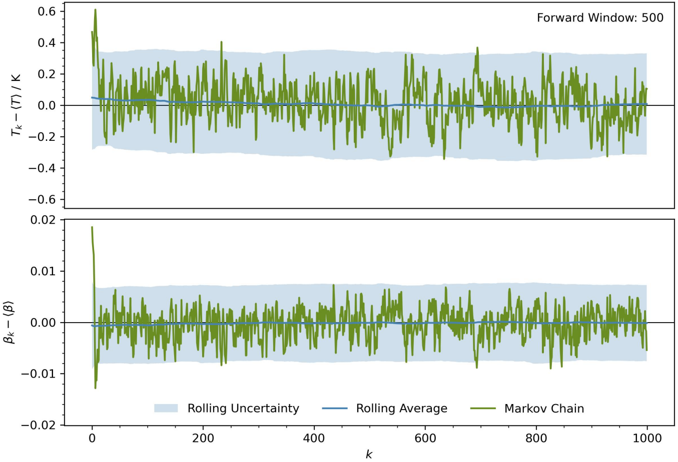
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$



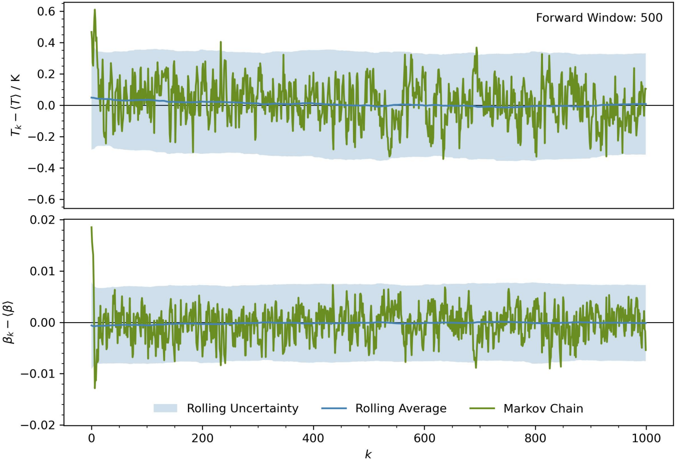
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
    - $h = 5 \cdot 10^{-3}$ & acc = 30.27%



Application

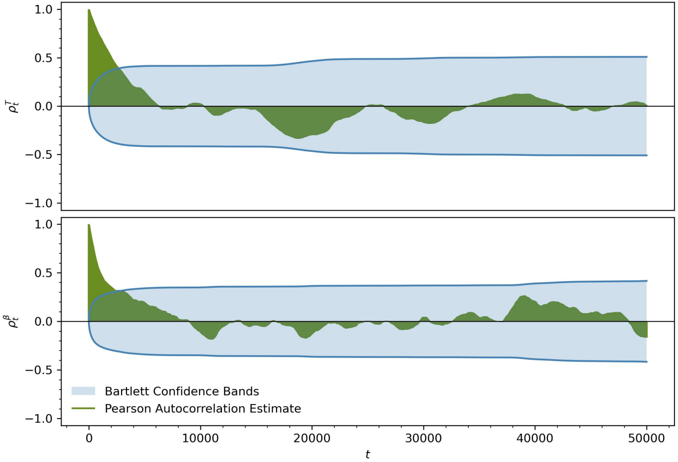
- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 5 \cdot 10^{-3}$ & acc = 30.27%
  - acceptance and step size seem good



Application

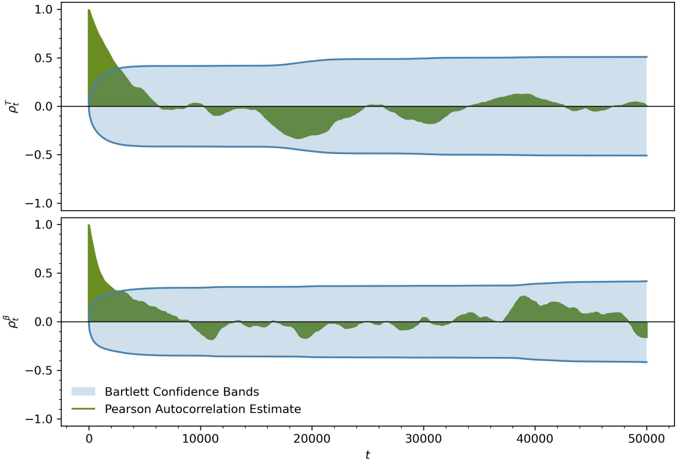
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$



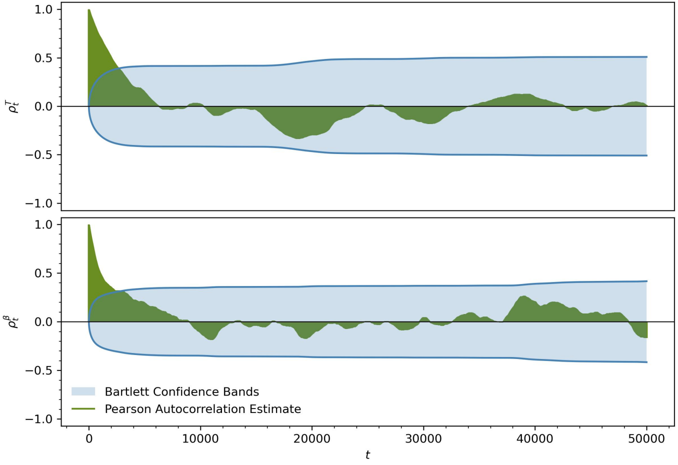
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-4}$ & acc = 97.72%



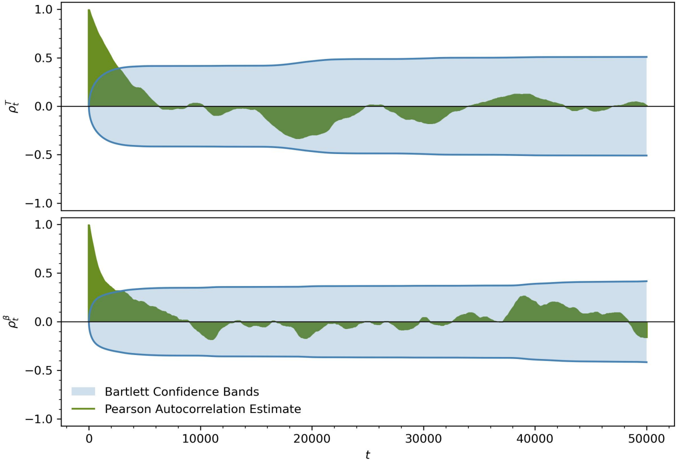
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-4}$ & acc = 97.72%
  - acceptance too high



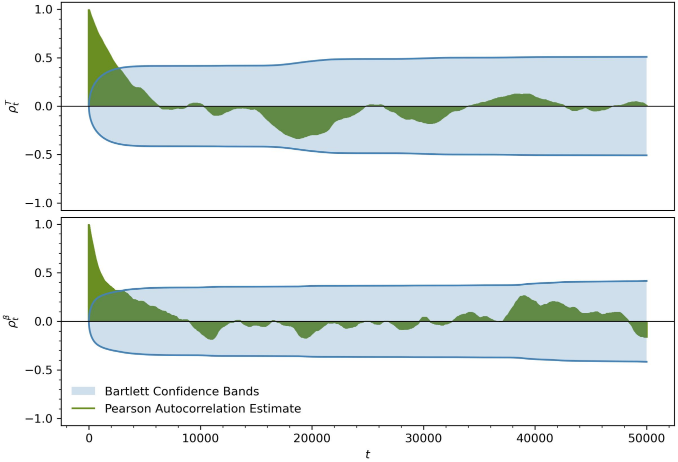
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-4}$ & acc = 97.72%
  - acceptance too high
  - step size too short



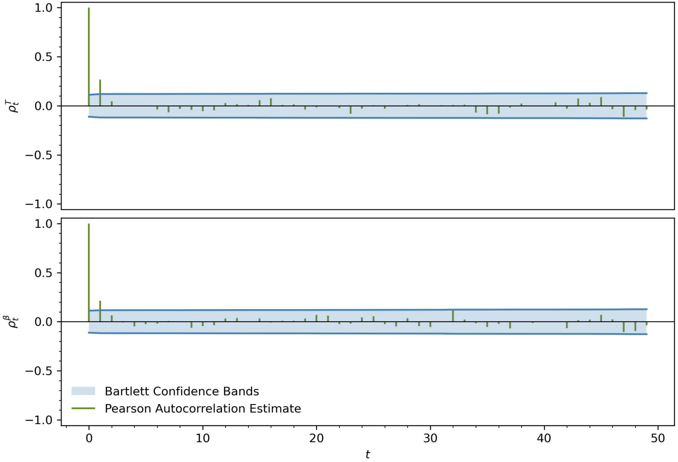
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-4}$ & acc = 97.72%
  - acceptance too high
  - step size too short
  - $l \approx 3000$



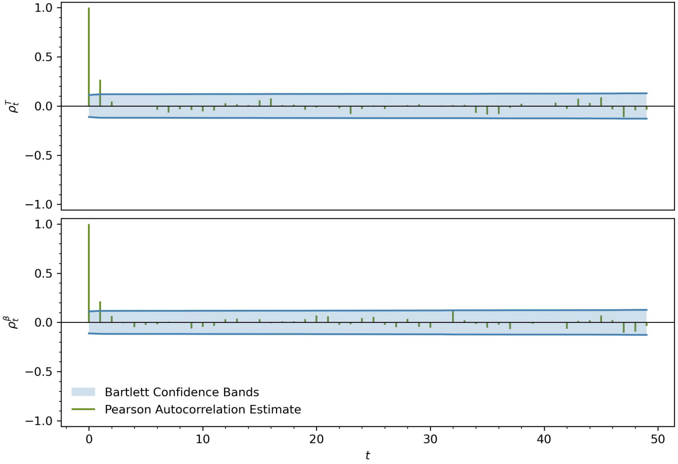
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$



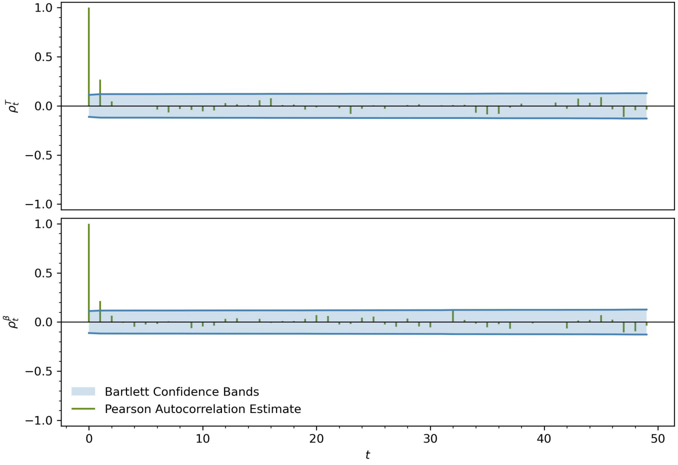
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-2}$ & acc = 12.13%



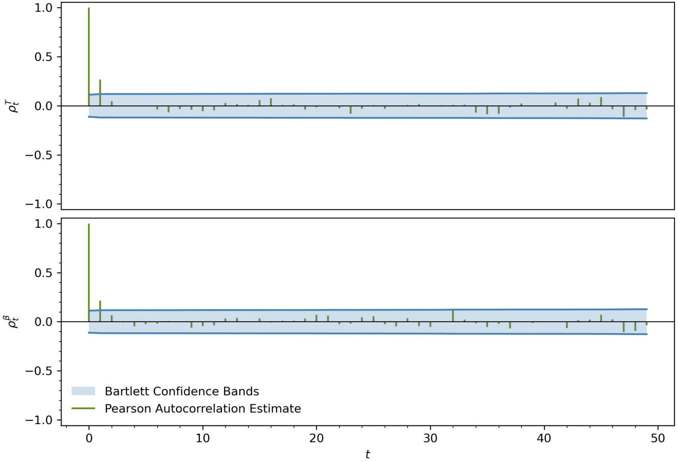
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-2}$ & acc = 12.13%
  - acceptance too low



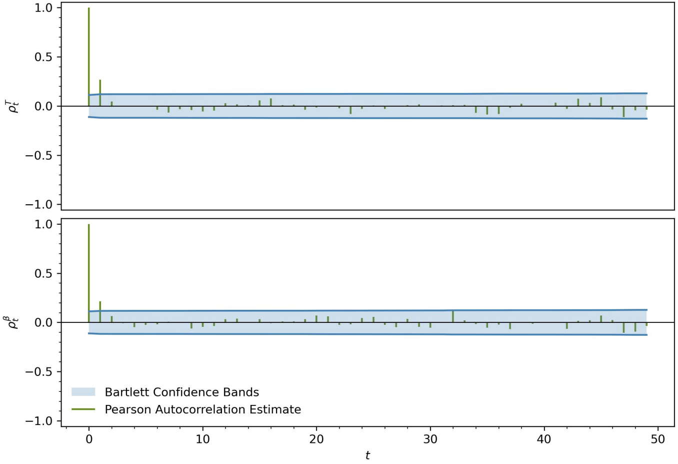
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-2}$ & acc = 12.13%
  - acceptance too low
  - step size too long



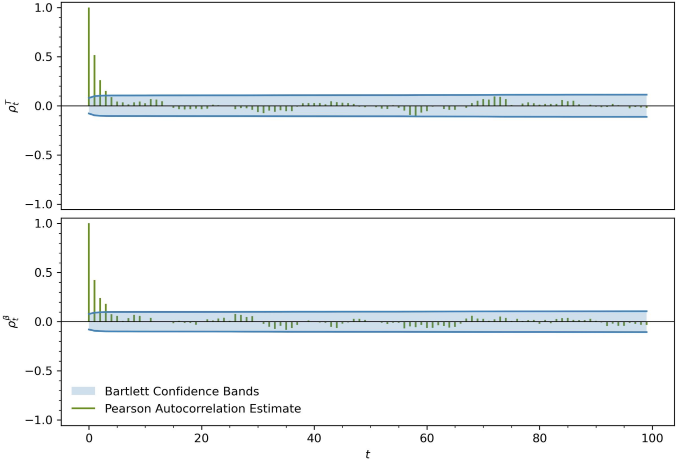
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 10^{-2}$ & acc = 12.13%
  - acceptance too low
  - step size too long
  - $l = 2$



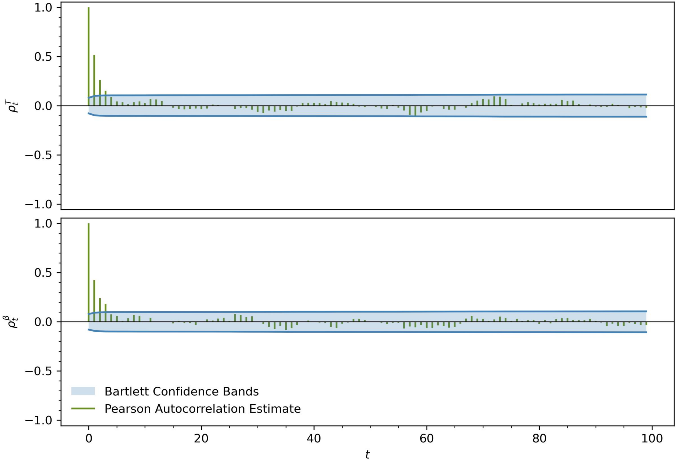
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$



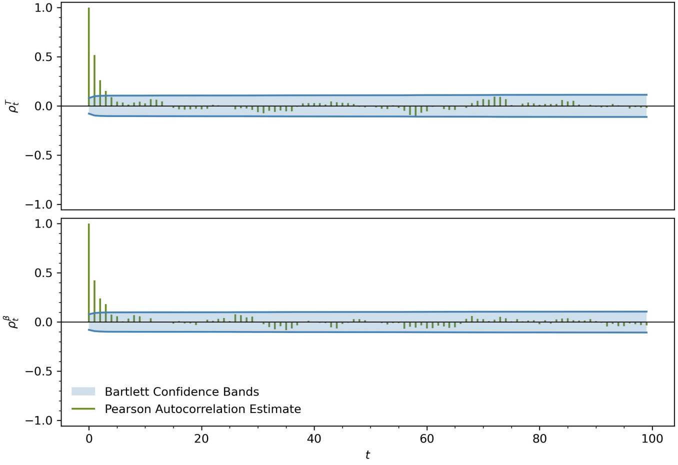
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 5 \cdot 10^{-3}$ & acc = 30.27%



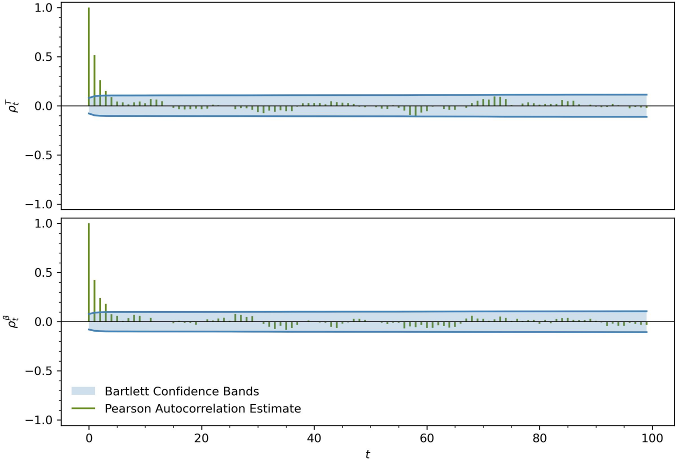
Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 5 \cdot 10^{-3}$ & acc = 30.27%
  - acceptance and step size seem good



Application

- *Metropolis Algorithm:* $x_j \sim N(x_i, h)$
  - $h = 5 \cdot 10^{-3}$ & acc = 30.27%
  - acceptance and step size seem good
  - $l = 4$



Overview

Shortcomings

Shortcomings

- *Expensive:* should only be used when absolutely necessary, do not do the following

Shortcomings

- *Expensive:* should only be used when absolutely necessary, do not do the following
  - sample the entire parameter space

Shortcomings

- *Expensive:* should only be used when absolutely necessary, do not do the following
  - sample the entire parameter space
  - improve poor optimisation performance

Shortcomings

- *Expensive:* should only be used when absolutely necessary, do not do the following
  - sample the entire parameter space
  - improve poor optimisation performance
  - discard distributions at intermediate steps

Shortcomings

- *Expensive:* should only be used when absolutely necessary, do not do the following
  - sample the entire parameter space
  - improve poor optimisation performance
  - discard distributions at intermediate steps
- *Other Tools:*

Shortcomings

- *Expensive:* should only be used when absolutely necessary, do not do the following
  - sample the entire parameter space
  - improve poor optimisation performance
  - discard distributions at intermediate steps
- *Other Tools:*
  - *Inversion Sampling:* transform uniform random numbers to desired distribution

Shortcomings

- *Expensive:* should only be used when absolutely necessary, do not do the following
  - sample the entire parameter space
  - improve poor optimisation performance
  - discard distributions at intermediate steps
- *Other Tools:*
  - *Inversion Sampling:* transform uniform random numbers to desired distribution
  - *Rejection Sampling:* propose samples, then accept or reject based on density

Shortcomings

- *Expensive:* should only be used when absolutely necessary, do not do the following
  - sample the entire parameter space
  - improve poor optimisation performance
  - discard distributions at intermediate steps
- *Other Tools:*
  - *Inversion Sampling:* transform uniform random numbers to desired distribution
  - *Rejection Sampling:* propose samples, then accept or reject based on density
  - *Importance Sampling:* weight samples from simple density to match target

Thank You!

- A. Casey, *Data Analysis and Machine Learning*[3] 2022
- M. Linhoff & W. Rhode, *Statistical Methods of Data Analysis* 2023
- *emcee documentation*[4] 2025

Resources on GitHub: here